



“ 소프트웨어로 하드웨어를 제어해 보자 ”

IOT 실습심화

7주차 : 스마트 도어락



7주차 수업은 스마트 도어락



HTTP Server ⟷ HTTP Client

응용 프로그래밍

IoT 제어 시스템 (project.py)

시스템 프로그래밍

Linux OS

Flask 웹 프레임워크

네트워크 TCP/IP

네트워크 프로그래밍

LED (RGB) | 서보 모터 | 부저 | LCD | 센서 | 카메라



RaspberryPi

python™

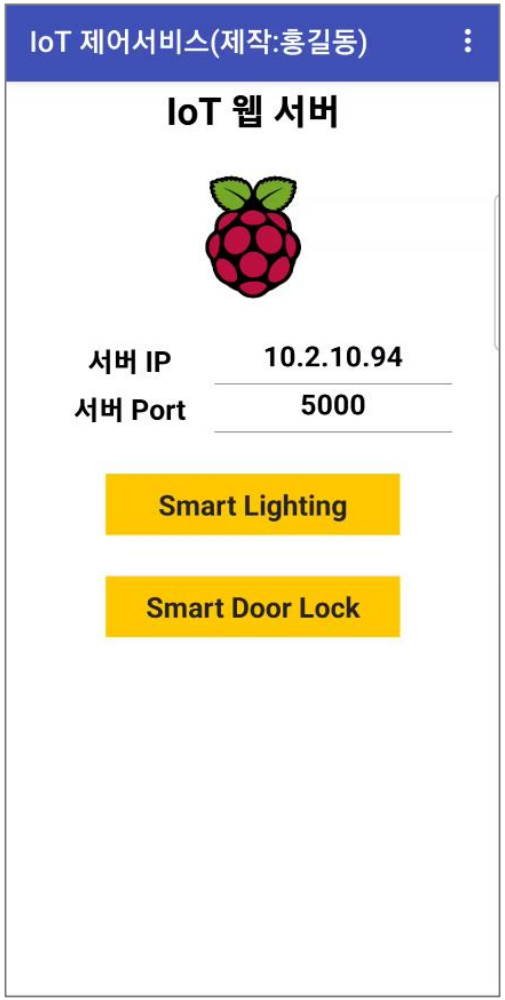
“ 소프트웨어로 하드웨어를 제어해 보자”

* IoT 기술 요소 : 센싱 기술, 네트워킹 기술, 서비스 인터페이스 기술

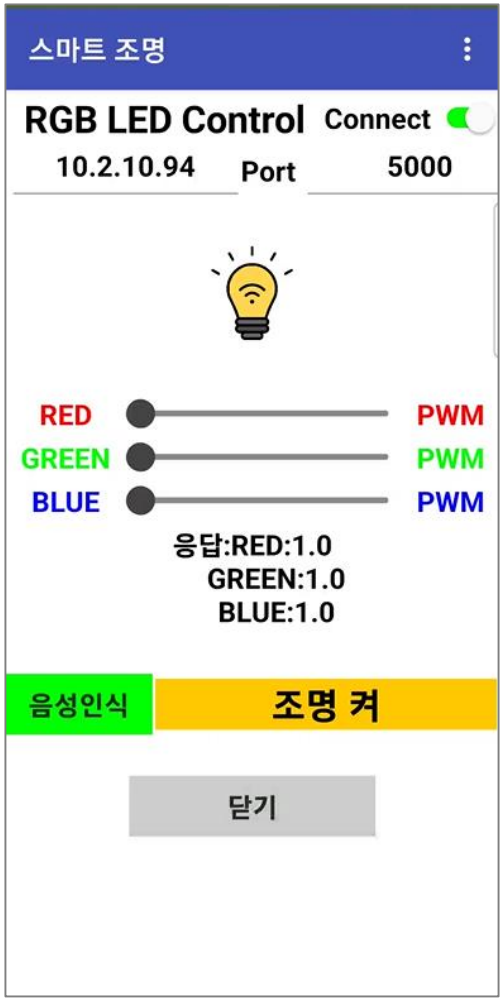
7주차 수업은 스마트 도어락

- 7주차 주제는 서보모터를 이용한 '스마트 도어락'

Screen1(홈 화면)



Screen2(스마트조명)



Screen3(스마트도어락)



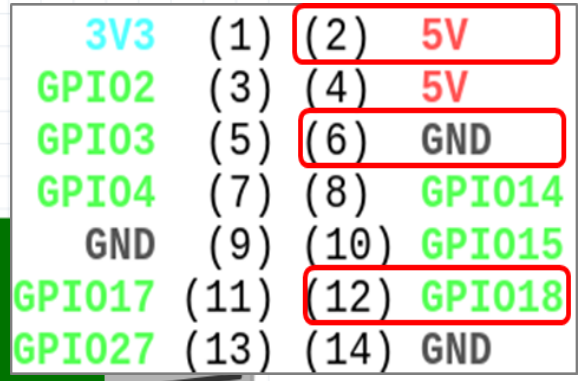
수업준비) 서보모터 정상동작 확인

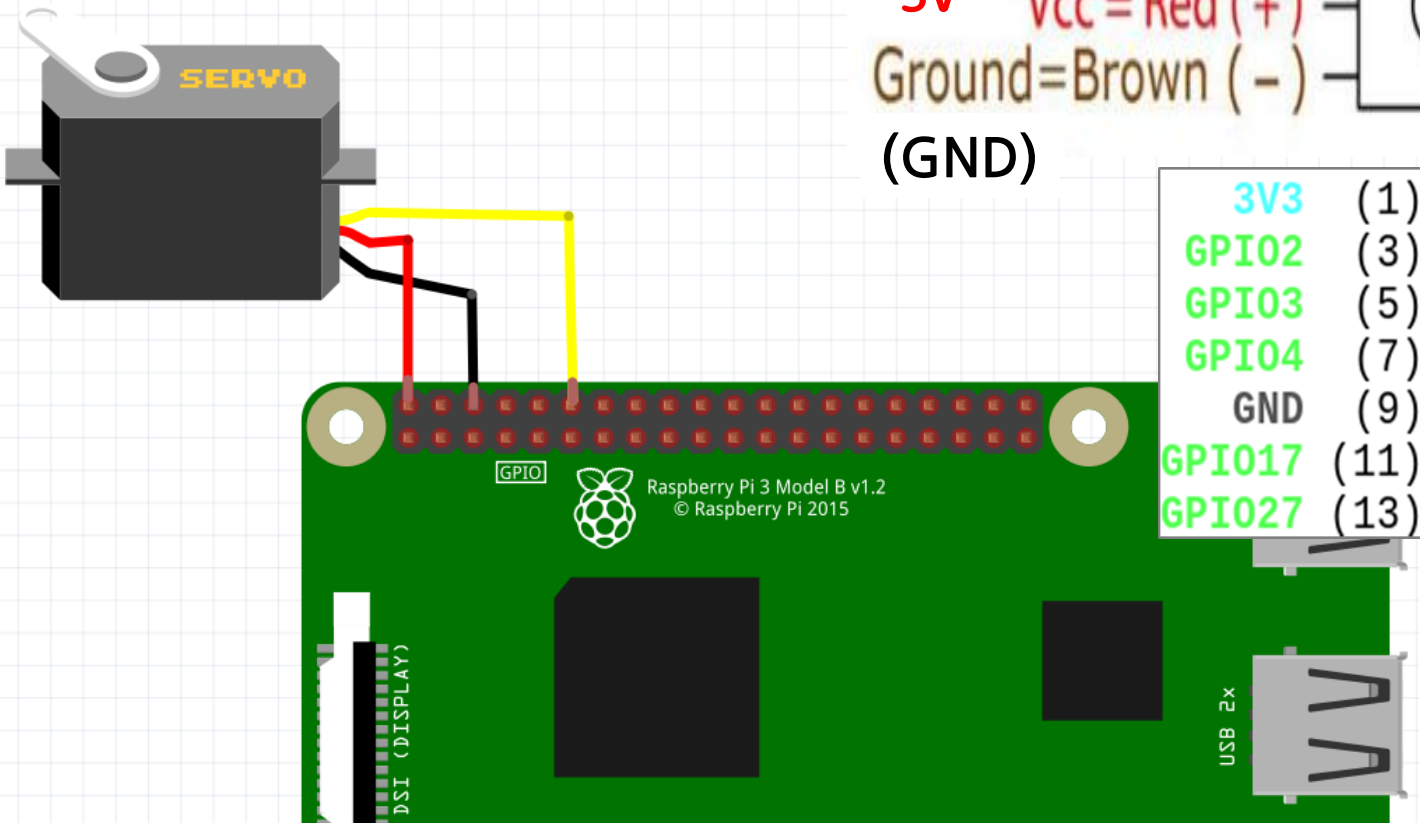
- 수업 시작 전에 servo.py 실행해서 서보모터의 정상동작 확인해 주세요

- 서보모터의 GPIO는 18번 사용 (GPIO18)



(GND)





수업준비) 서보모터 정상동작 확인

- 수업 시작 전에 servo.py 실행해서 서보모터의 정상동작 확인해 주세요

- 서보모터 방향제어 핵심 로직

servo.py

```
# PWM으로 서보모터 방향 제어하기
from gpiozero import Servo
import time

# SG90 서보모터 객체 생성(GPIO18번), PWM 조정값 적용
servo = Servo(18, min_pulse_width=0.5/1000, max_pulse_width=2.5/1000)

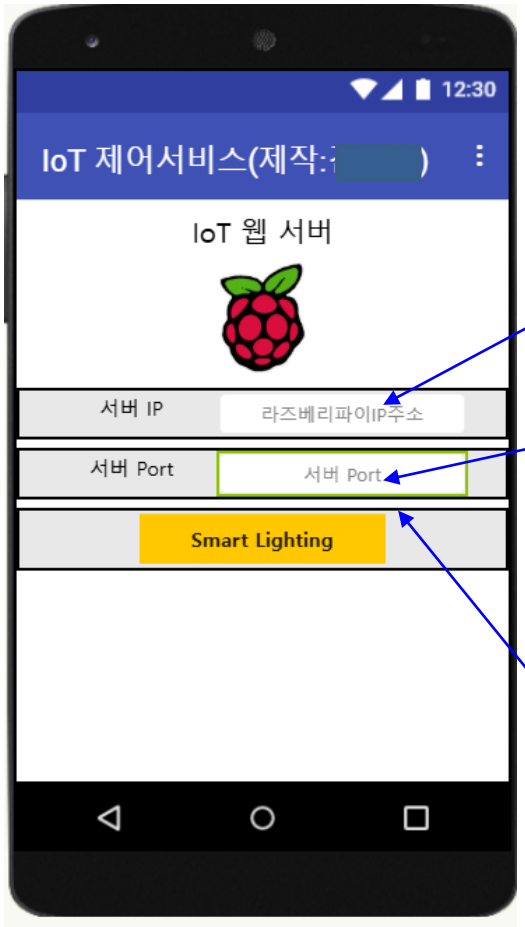
print("서보모터 위치제어 실습(전선을 정면으로 보기)")
print('='*45)
print("1:left(0도), c:center(90도), r:right(180도)")
print("프로그램 종료는 'q'입력")
print('='*45)
```

```
try:
    while True:
        # 사용자로 위치 입력받기
        command = input("원하는 위치를 선택하세요(l/d/r/q)")
        if command == 'l':
            print("Left 위치로 이동 (value: -1.0)")
            servo.min()
            #servo.value = -1.0   #동일하게 작동
        elif command == 'c':
            print("Center 위치로 이동 (value: 0.0)")
            servo.mid()
            #servo.value = 0.0    # 동일하게 작동
        elif command == 'r':
            print("Right 위치로 이동 (value: 1.0)")
            servo.max()
            #servo.value = 1.0     # 동일하게 작동
        elif command == 'q':
            print("프로그램을 종료합니다")
            break
        else:
            print("잘못된 입력입니다")
        # 모터가 해당위치까지 물리적으로 회전할 최소한의 시간 주기
        time.sleep(0.5)
        servo.detach()  # 이동 후 펄스신호를 끊어 떨림(jitter) 방지
except KeyboardInterrupt:
    pass
finally:
    servo.close()
```

6주차 스마트조명 결과물 리뷰

❑ 6주차 스마트조명 결과물 리뷰

Screen1



IP 입력과 Port 입력 텍스트박스의 텍스트에 미리 자신의 라즈베리파이 IP주소와 5000 번을 속성에 적용해 놓읍시다!

텍스트박스	텍스트박스_IP입력	글꼴굵게, 글꼴크기-15, 너비 - 50% 텍스트 - 라즈베리파이IP주소, 텍스트정렬- 가운데
텍스트박스	텍스트박스_Port 입력	글꼴굵게, 글꼴크기-15, 너비 - 50%, 숫자만, 텍스트 - 5000, 텍스트정렬- 가운데

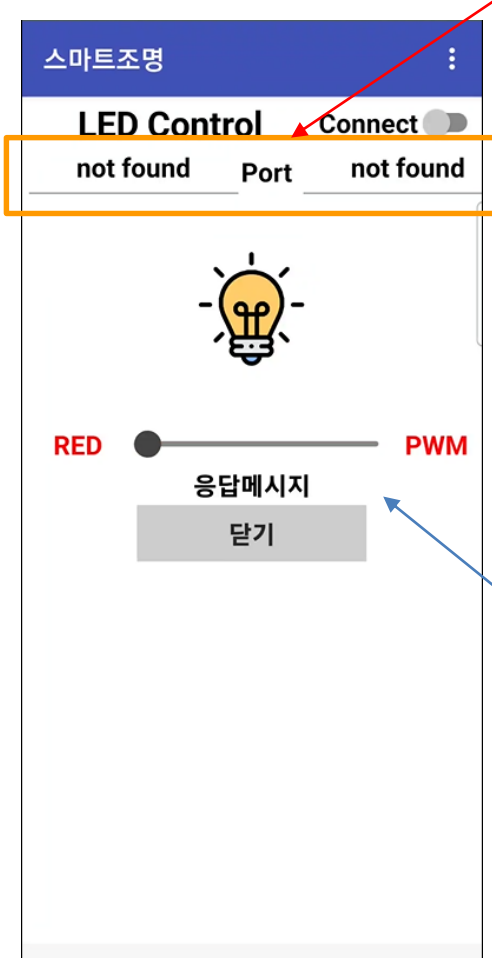
디자인 하면서 여백을 적절히 적용해 주세요!

6주차 스마트조명 결과물 리뷰

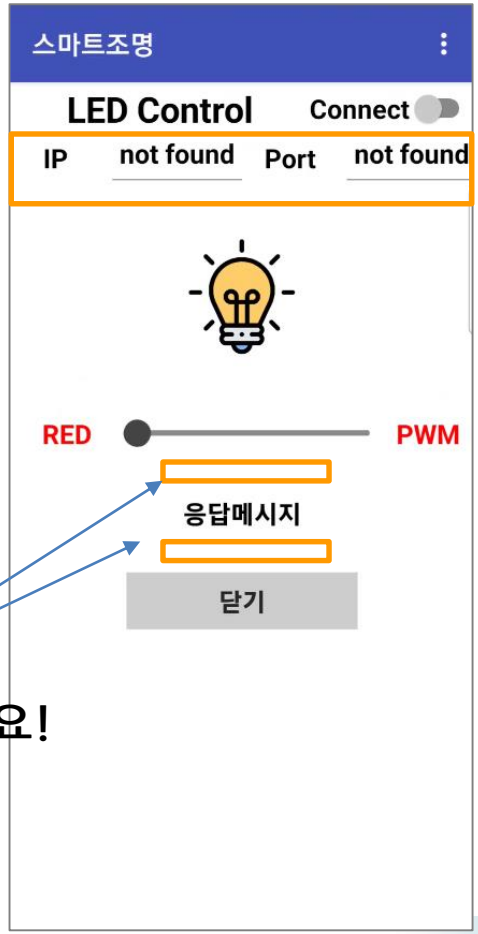
❑ 6주차 스마트조명 결과물 리뷰

Screen2

IP 레이블이 폰 화면에 보이지 않습니다
따라서 컴포넌트의 속성에서 너비의 비율을 적절히 적용해 줍시다.



수평배치	수평배치1	수평, 수직정렬-가운데, 너비-부모요소
레이블	레이블1_IP	글꼴굵게, 글꼴크기-15, 텍스트-IP, 너비 - 20%, 정렬-가운데
텍스트박스	텍스트박스1_IP	글꼴굵게, 글꼴크기-15, 너비- 30%, 정렬- 가운데
레이블	레이블1_Port	글꼴굵게, 글꼴크기-15, 텍스트- Port, 너비-20%, 정렬-가운데
텍스트박스	텍스트박스1_Port	글꼴굵게, 글꼴크기-15, 너비-30%, 정렬- 가운데, 숫자만



디자인 하면서 여백을 적절히 적용해 주세요!

6주차 스마트조명 결과물 리뷰

❑ 6주차 스마트조명 결과물 리뷰

클라이언트 앱은
사용자 요청사항을 POST 방식으로 요청메시지 body에 텍스트로 전송합니다



* 주의) 파일POST 전송하기 → 텍스트 POST로 전송하기

6주차 스마트조명 결과물 리뷰

- 6주차 스마트조명 결과물 리뷰

앱 인벤터에서 슬라이더 컴포넌트의 섬네일 값이 실수로 전달되는 경우(아이폰)
서버측에서 에러가 발생하는 문제 해결

```
# CSR방식. POST 요청에 대한 처리(앱 인벤터 제어용)
@app.route('/post', methods=['POST'], strict_slashes=False)
def post_RGB():
    # 요청메시지 바디에서 key로 value 저장
    color = request.form.get('color')
    pwm_v = request.form.get('pwm_value')

    RGB_state_dict[color] = int(pwm_v)/100
    update_RGB() # RGBLED pwm 값 업데이트
    # 앱 인벤터가 성공여부를 확인할 수 있도록 결과 문자열 반환
    return f"""RED:{RGB_state_dict['RED']}
       GREEN:{RGB_state_dict['GREEN']}
       BLUE:{RGB_state_dict['BLUE']}"""
```

```
float(pwm_v)/100
```

float으로 형변환하기

다양한 IoT 서비스를 제작하는 방법

- 소자가 변경되더라도 IoT 서비스의 처리 로직은 동일

소자에 따라 제어하는 로직과
클라이언트의 요구사항이 다를 뿐
기본적으로 클라이언트의 파라미터(요구사항)을 받아서 웹 서버에서
처리하는 로직은 공통입니다.

따라서 처음부터 새롭게 구현하지 말고
이전 결과물에서 재활용할 수 있는 부분은 최대한 재활용합시다!

IoT 서비스의 처리 로직은 동일

❑ 소자가 변경될 때 마다 코드상 변경되는 부분

- Flask 웹 서버 코드

```
# 스마트조명 웹 서버 프로그램
from flask import Flask, request, render_template
from gpiozero import RGBLED

# 웹 서버 객체 생성
app = Flask(__name__)

# RGBLED pin설정 및 객체 생성
rgb = RGBLED(red=17, green=22, blue=27)

# RGB 상태 DB (딕셔너리)
RGB_state_dict = {'RED': 0, 'GREEN': 0, 'BLUE':0}

# RGBLED 제어 핵심로직
def update_RGB():
    r_val = float(RGB_state_dict['RED'])
    g_val = float(RGB_state_dict['GREEN'])
    b_val = float(RGB_state_dict['BLUE'])
    rgb.color = (r_val, g_val, b_val)

update_RGB()  # RGBLED 초기화
```

소자 제어 핵심로직부

```
# 두드도 접속 시 처리
@app.route('/', strict_slashes=False)
def index():
    return render_template('RGB_index.html', RGB_states=RGB_state_dict)

# SSR방식. GET 요청에 대한 처리 (웹 브라우저용)
@app.route('/get', methods=['GET'], strict_slashes=False)
def get_RGB():
    # URL 쿼리에서 key로 value 저장하기
    color = request.args.get('color')
    state = request.args.get('state')

    RGB_state_dict[color] = int(state)
    update_RGB()   # RGBLED 상태 업데이트
    return render_template('RGB_index.html', RGB_states=RGB_state_dict)

# CSR방식. POST 요청에 대한 처리(앱 인벤터 제어용)
@app.route('/post', methods=['POST'], strict_slashes=False)
def post_RGB():
    # 요청메시지 바디에서 key로 value 저장
    color = request.form.get('color')
    pwm_v = request.form.get('pwm_value')

    RGB_state_dict[color] = int(pwm_v)/100
    update_RGB() # RGBLED pwm 값 업데이트
    # 앱 인벤터가 성공여부를 확인할 수 있도록 결과 문자열 반환
    return f"""RED:{RGB_state_dict['RED']}
       GREEN:{RGB_state_dict['GREEN']}
       BLUE:{RGB_state_dict['BLUE']}"""

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000)
```

소자에 맞는 웹 페이지 제공부분

클라이언트 요구사항 받아서 처리하는 부분

클라이언트 요구사항 받아서 처리하는 부분

IoT 서비스의 처리 로직은 동일

- 소자가 변경될 때 마다 코드상 변경되는 부분

- HTML 코드

```
<!DOCTYPE html>
<html>
<head>
    <!-- 스마트폰 화면 크기에 맞춰 웹페이지를 맞춰주는 필수 태그 -->
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>LED</title>
</head>
<body>
    <h3>Smart LED Control Server</h3>

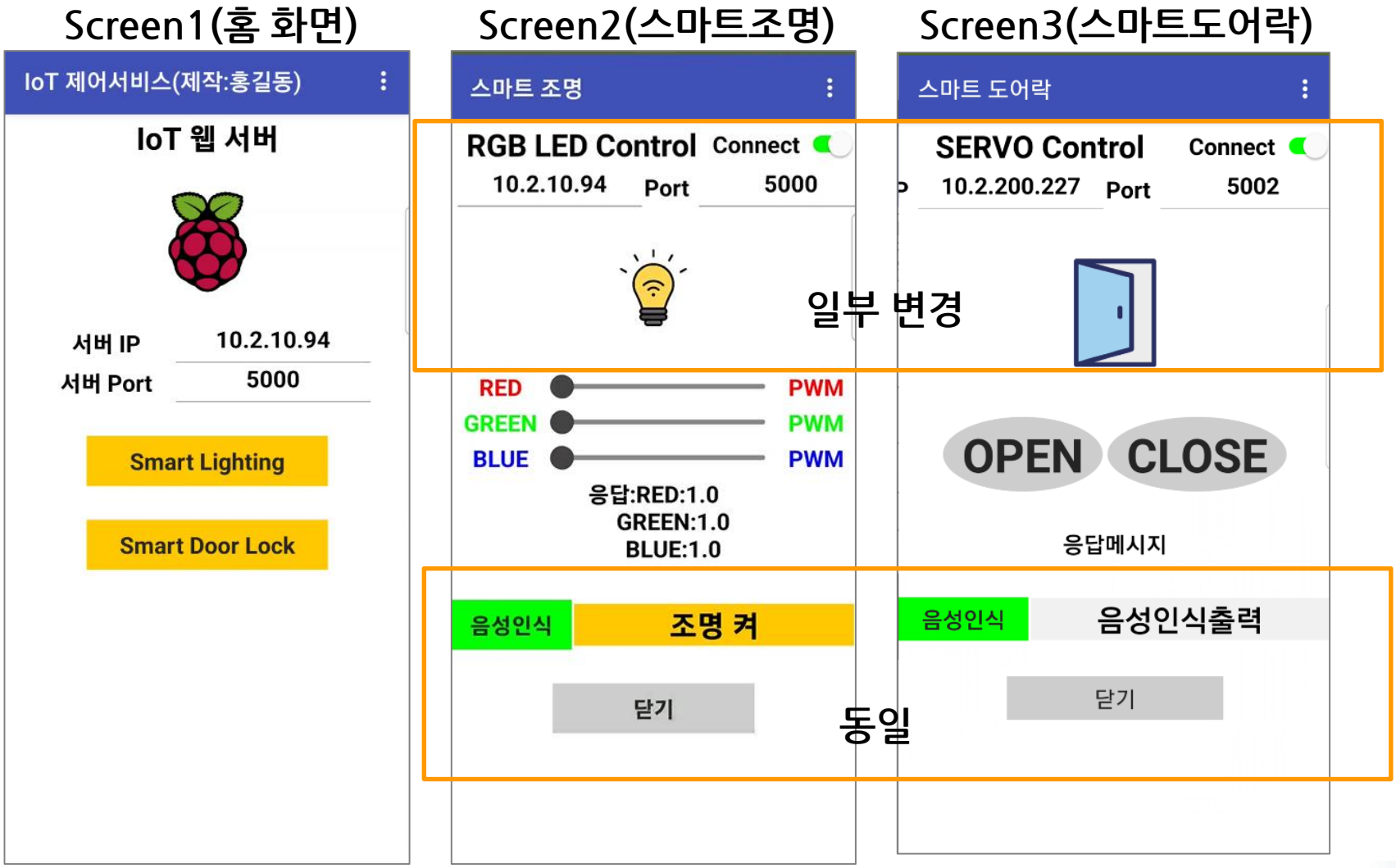
    <!-- Jinja2 반복문 시작 -->
    {% for color, state in RGB_states.items() %}
    <p>
        <!-- 딕셔너리의 key(색상)과 value(상태)를 받아 화면에 출력 -->
        {{ color }} : {{ state }}
        <br>
        <!-- 버튼 클릭시 주소창에 파라미터를 담아 서버로 전송 -->
        <a href="/get?color={{ color }}&state=1"><button>ON</button></a>
        <a href="/get?color={{ color }}&state=0"><button>OFF</button></a>
    </p>
    {% endfor %}
    <!-- jinja2 반복문 끝 -->

</body>
</html>
```

클라이언트에게
제공하는 폼데이터(양식)

IoT 서비스의 처리 로직은 동일

- 소자가 변경될 때 마다 코드상 변경되는 부분 - 클라이언트 앱



1. 스마트조명: 음성으로 제어하기

❑ 도전과제 (Stretch Goal)

스마트 폰과 조명의 만남 → IoT 기술 기반의 스마트 조명

- **조명의 밝기, 색상등을 원격으로 제어 가능**
- **기술적 기반위에서 다양한 기능 구현 가능**

  - 자동화 설정 : 특정 시간에 조명 제어
  - 음성 제어 : 음성 명령을 통해 제어
  - 알림 기능: 전화, 문자등 알림이 오는 경우 조명 제어

이 중에서 음성명령을 통해 조명을 원격으로 제어해 봅시다

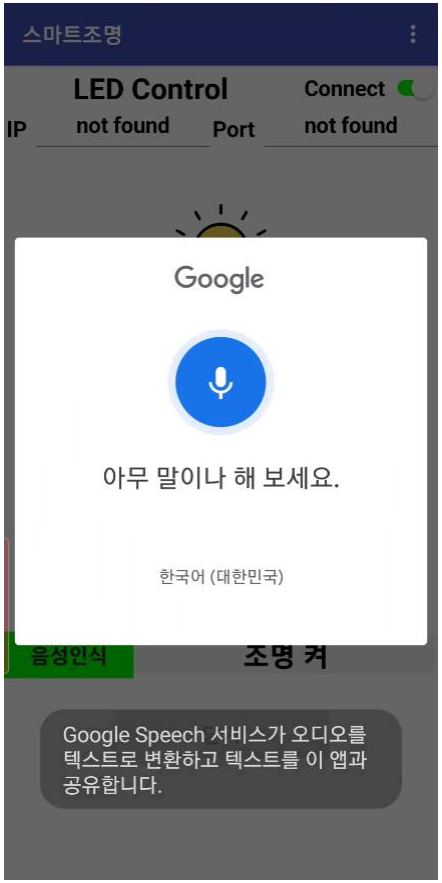
1. 스마트조명: 음성으로 제어하기

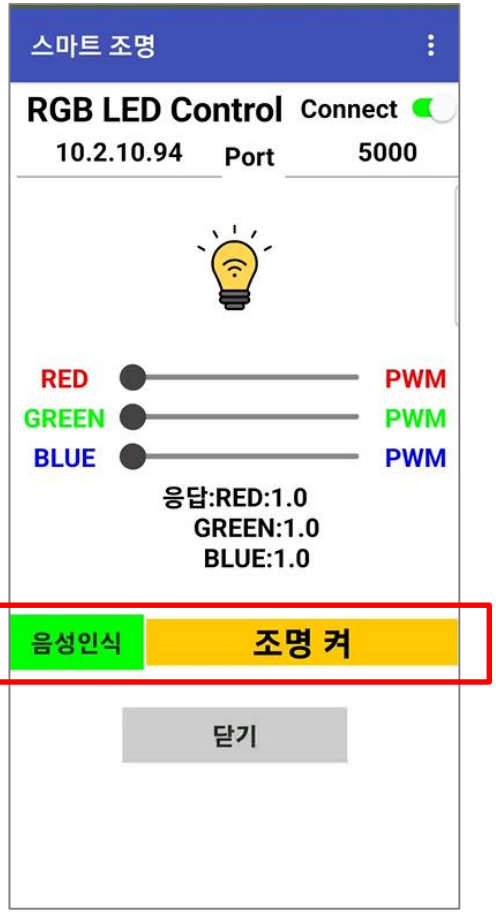
❑ 음성 명령을 통해 조명 제어

스마트조명은 음성 명령을 통해 제어가 가능합니다.
[음성인식] 기능을 통해 스피커로 "조명 켜!“, “조명 꺼!”와 같이 말하면 제어가 되도록 구현해 봅시다

조명 켜!







1. 스마트조명: 음성으로 제어하기

❑ 음성 명령을 통해 조명 제어 - IoT 웹 서버와 클라이언트 앱 보완하기

[구현 아이디어]

- RGB PWM을 서버에서 한 번에 제어하기 위한 아이디어가 필요 (예: ALL)
- 앱 코드와 서버간 연동해서 상호작용하도록 구현 필요
- 어떻게 하면 코드의 변경을 최소화하면서 구현할 것인가?

1. 스마트조명: 음성으로 제어하기 - 앱 디자인

❑ 음성 명령을 통해 조명 제어 - 앱 디자인 추가하기(6주차 앱 프로젝트에 추가)



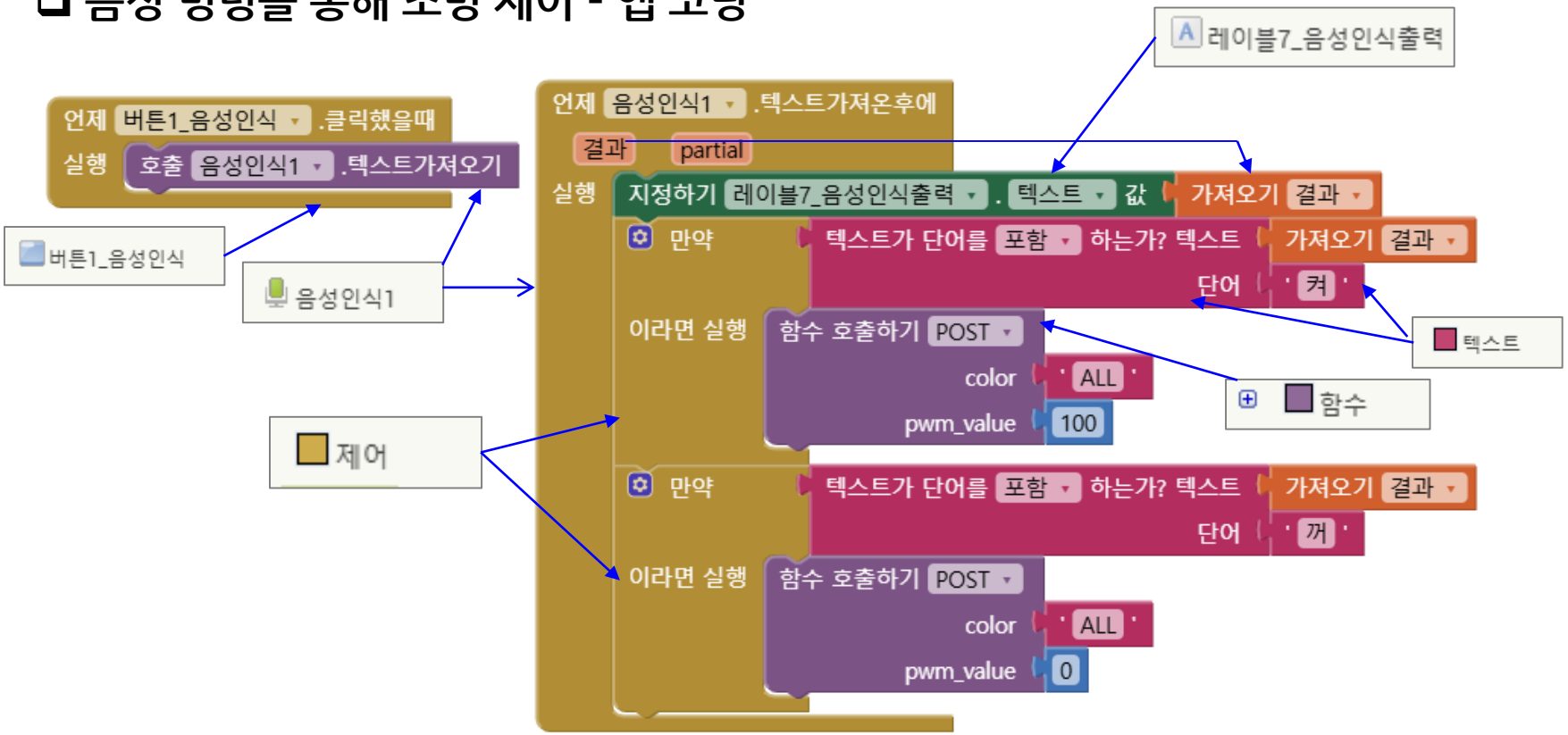
Screen2에 추가하기

	컴포넌트	이름	속성
①	수평배치	수평배치1	수평, 수직정렬-가운데, 너비-부모요소
	버튼	버튼1_음성인식	배경색-초록, 글꼴굵게, 글꼴크기-15, 너비 - 30%, 모양-직사각형, 텍스트-음성인식, 텍스트정렬- 가운데
	레이블	레이블1_음성인식 출력	배경색-밝은회색, 글꼴굵게, 글꼴크기-20, 너비-부모요소, 텍스트 - 음성인식출력, 정렬-가운데
②	음성인식	음성인식1	

※ 음성인식 컴포넌트는 팔레트 >> 미디어 >> 음성인식

1. 스마트조명: 음성으로 제어하기 - 앱 코딩

❑ 음성 명령을 통해 조명 제어 - 앱 코딩



[구현 아이디어]
- RGB PWM을 서버에서 한 번에 제어하기 위한 아이디어가 필요 (예: 'ALL' 사용)
- 명령에 '켜' 라는 단어가 있으면 POST 함수를 이용해서 RGB LED 모두 100% 작동
  body form data = {'color' : 'ALL', 'pwm_value' : 100}
- 명령에 '꺼' 라는 단어가 있으면 POST 함수를 이용해서 RGB LED 모두 0% 작동
  body form data = {'color' : 'ALL', 'pwm_value' : 0}

1. 스마트조명: 음성으로 제어하기 - 웹 서버

❑ CSR Client-Side Rendering 방식으로 라즈베리파이 웹 서버 구현

- RGB_app.py에서 POST 요청 처리 부분을 보완해 보세요
  - 클라이언트가 전달하는 'ALL'을 받아서 처리하는 코드를 추가해 보세요

6주차 코드

```
# 2. CSR방식.  POST 요청 처리 (앱 인벤터 제어용)
@app.route('/post', methods=['POST'], strict_slashes=False)
def post_RGB():
    color = request.form.get('color')
    pwm_v = request.form.get('pwm_value')

    RGB_state_dict[color] = int(pwm_v)/100
    update_RGB()

    # 앱 인벤터가 성공 여부를 확인할 수 있도록 결과 문자열을 반환
    return f"""RED:{RGB_state_dict['RED']},
        GREEN:{RGB_state_dict['GREEN']},
        BLUE:{RGB_state_dict['BLUE']}"""
```

코드변경

```
# CSR방식. POST 요청에 대한 처리(앱 인벤터 제어용), 음성제어 추가
@app.route('/post', methods=['POST'], strict_slashes=False)
def post_RGB():
    # 요청메시지 바디에서 색상(key)과 해당하는 pwm(value) 추출
    color = request.form.get('color')
    pwm_v = request.form.get('pwm_value')
    val = float(pwm_v)/100.0    # pwm 듀티비로 변환

    #  색상 정보가 "ALL" 경우 pwm_v값 추출해서 모든 LED에 적용
    if color == 'ALL':
        # for 문을 통해 모든 색상에 동일한 값 저장
        For문을 이용해
        모든 색상(키)에 PWM(값) 적용

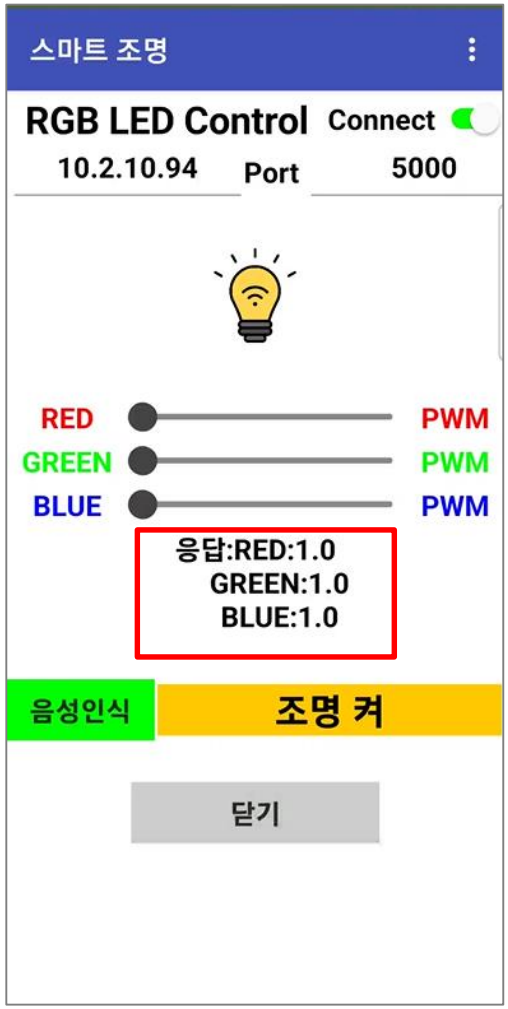
    # 단일 색상 요청의 경우
    elif color in RGB_state_dict:
        pwm_v = request.form.get('pwm_value')
        RGB_state_dict[color] = val

    update_RGB() # RGBLED pwm 값 업데이트

    # 앱 인벤터가 성공여부를 확인할 수 있도록 결과 문자열 반환
    return f"""RED:{RGB_state_dict['RED']}
       GREEN:{RGB_state_dict['GREEN']}
       BLUE:{RGB_state_dict['BLUE']}"""
```

1. 스마트조명: 음성으로 제어하기

❑ 음성 명령을 통해 조명 제어 - 테스트

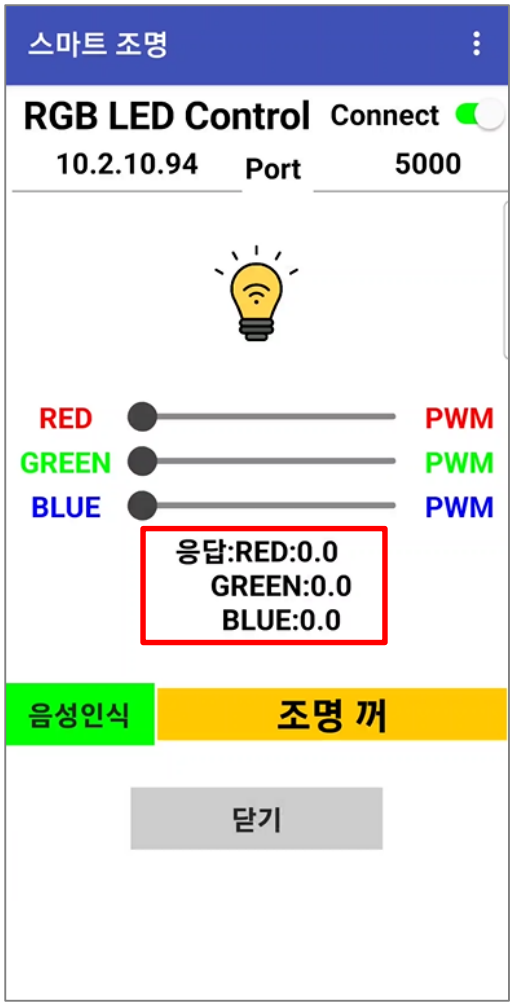


조명 켜!

[동작확인]
RGB 모두 100% 작동

[동작확인]
RGB 모두 0% 작동

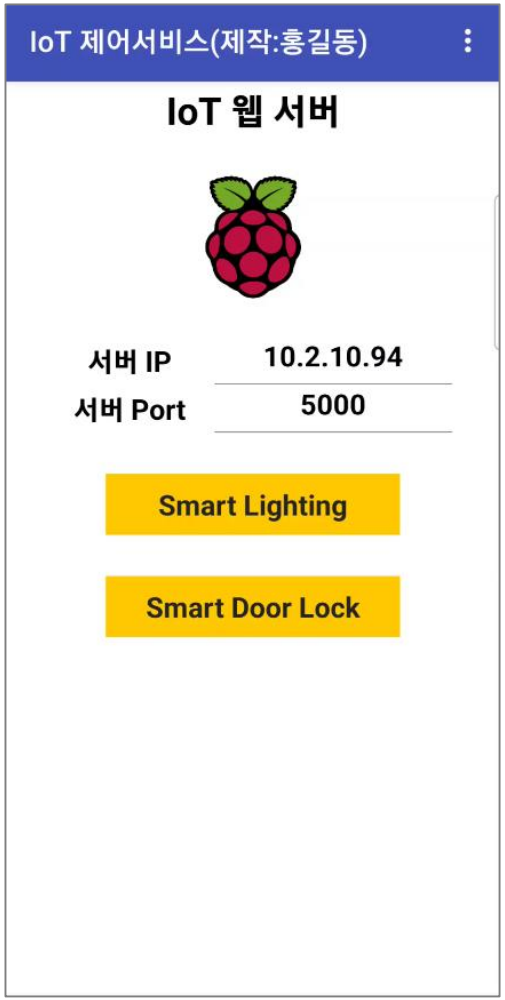
조명 꺼!



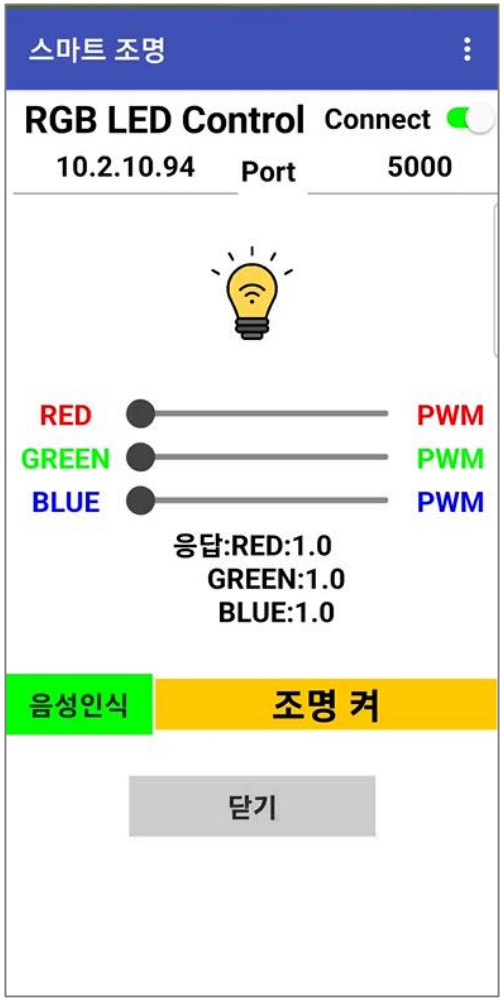
7주차 주제는 스마트 도어락

- 7주차 주제는 서보모터를 이용한 '스마트 도어락'

Screen1(홈 화면)



Screen2(스마트조명)



Screen3(스마트도어락)



2. 스마트 도어락



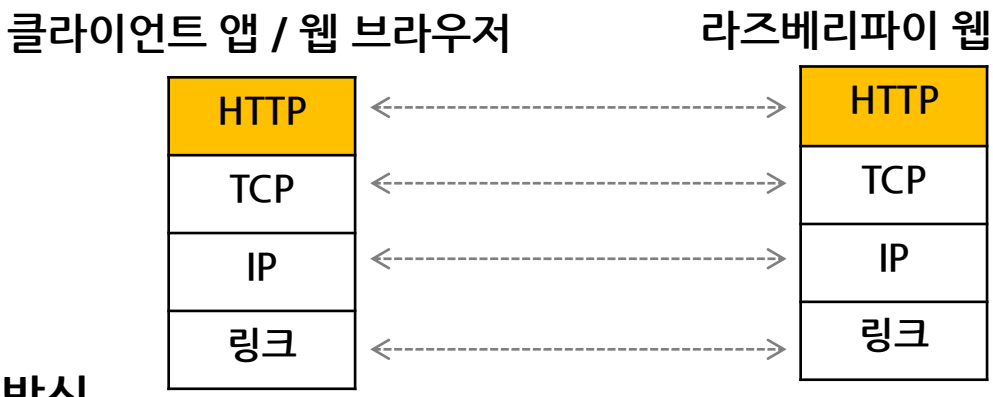
"7주차에는 "스마트 도어락" 앱을 제작하여
앱으로 라즈베리파이에 연결된
서보 모터의 방향을 제어해 봅시다"



2. 스마트 도어락 - 동작환경

❑ 스마트 도어락 프로토콜 스택

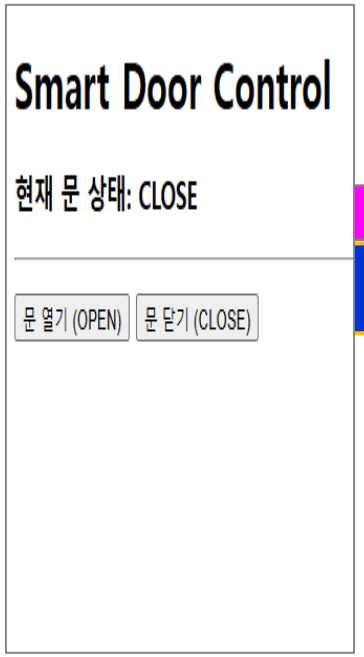
TCP/IP 프로토콜 스택

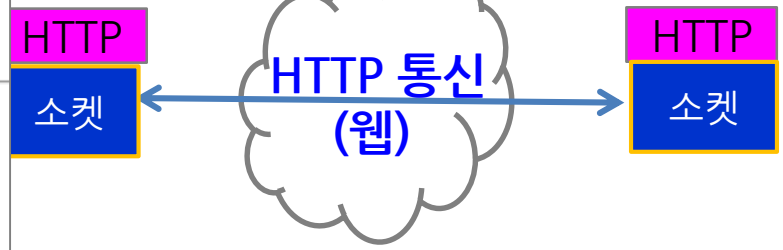


CSR방식

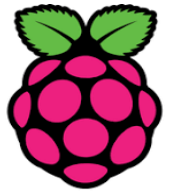


SSR방식









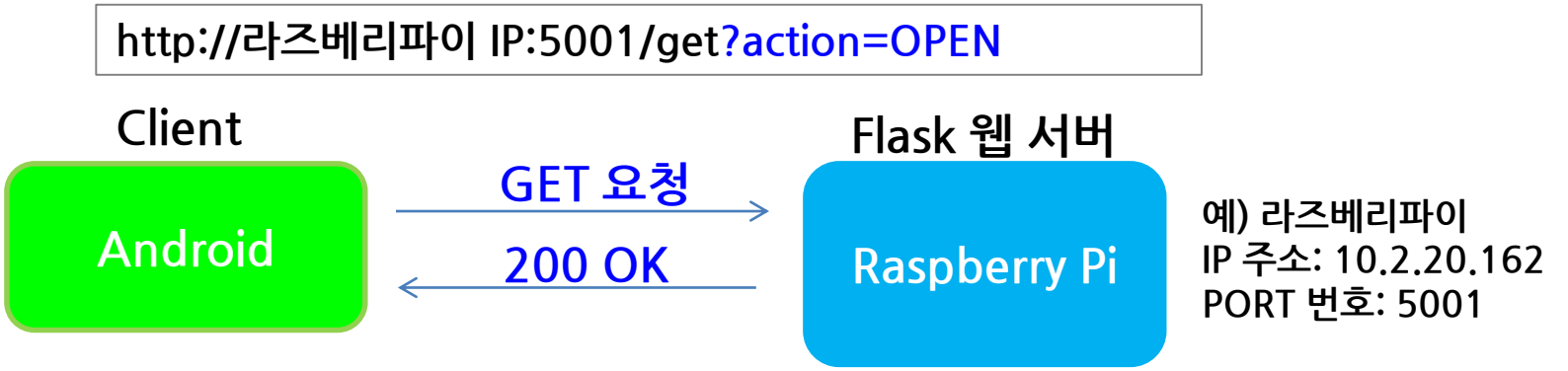
RaspberryPi

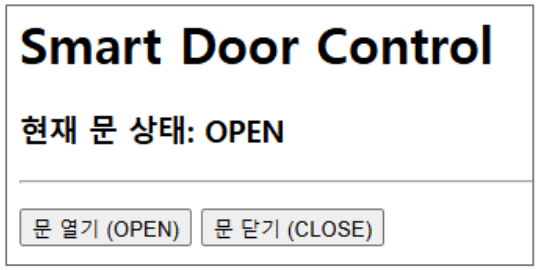
2. 스마트 도어락 - SSR 방식

❑ SSR Server-Side Rendering 방식으로 라즈베리파이 웹 서버 구현

◦ [동작 개요]

- **SSR 방식** : 서버가 데이터가 반영된 완성된 HTML 페이지를 만들어서 브라우저에 전달하는 방식
- **GET 방식 사용** : 클라이언트가 파라미터를 URL 쿼리스트링으로 보내 요청하는 방식





프로젝트 폴더/
├── door_app.py (Flask 서버 코드)
└── templates/
└── door_index.html (Jinja를 사용할 HTML 파일)

2. 스마트 도어락 - SSR 방식

❑ SSR Server-Side Rendering 방식으로 라즈베리파이 서버 구현 - GET 방식

◦ [동작 절차]

② http://라즈베리파이 IP:5001/get?action=OPEN

Client

Android

GET 요청

200 OK

Flask 웹 서버

Raspberry Pi

예) 라즈베리파이
IP 주소: 10.2.20.162
PORT 번호: 5001

① Smart Door Control

현재 문 상태: OPEN

문 열기 (OPEN) 문 닫기 (CLOSE)

③ query string paring

{'action': 'OPEN'}

④ action = request.args.get('action')

⑤ # 클라이언트 정보를 받아 PWM으로 서보모터 방향 제어
def door_control(action):

2. 스마트 도어락 - SSR 방식: 웹 서버 코드

❑ SSR Server-Side Rendering 방식으로 라즈베리파이 웹 서버 구현

◦ 이름_7주차_door_app.py

```python
# 스마트 도어락 제어 서버
from flask import Flask, request, render_template
from gpiozero import Servo
import time

# 웹 서버 객체 생성
app = Flask(__name__)

# SG90 서보모터 객체 생성 (GPIO 18번)
servo = Servo(18, min_pulse_width=0.5/1000, max_pulse_width=2.5/1000)

# door의 현재 상태를 저장할 전역변수 ('CLOSE' 또는 'OPEN')
current_status = 'CLOSE'

def control_door(action):
    global current_status
    if action == 'OPEN':
        servo.max()  # 서보모터를 최대각도(1.0)로 이동 -> 문열림
        current_status = 'OPEN'
    elif action == 'CLOSE':
        servo.min()  # 서보모터를 최소 각도(-1.0)로 이동 -> 문 닫힘
        current_status = 'CLOSE'
    # 모터가 회전할 수 있는 시간 주기
    time.sleep(0.5)
    # 이동 후 펄스신호를 끊어 떨림(jitter) 방지
    servo.detach()

# 초기 상태 설정 : 서버가 켜질때 문닫음
control_door('CLOSE')
```

```python
# 1. Flask 홈 화면 및 SSR 랜더링
@app.route('/', strict_slashes=False)
def index():
    # 현재 문 상태를 'door_status'라는 변수로 HTML로 전달
    return render_template('door_index.html', door_status=current_status)

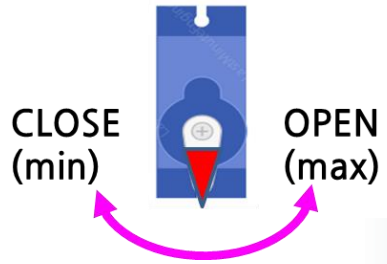
# 2. SSR방식. GET 요청 시 처리 (웹 버튼 클릭 시)
@app.route('/get', methods=['GET'], strict_slashes=False)
def get_door():
    action = request.args.get('action') # 주소창에서 ?action=값 읽기

    if action in ['OPEN', 'CLOSE']:
        control_door(action)

    return render_template('door_index.html', door_status=current_status)

if __name__ == "__main__":
    # 앞선 실습과 충돌나지 않도록 포트는 5001번으로 수정
    app.run(host='0.0.0.0', port=5001)
```

템플릿엔진으로 넘겨주는 값



2. 스마트 도어락 - SSR 방식: HTML 코드

❑ SSR Server-Side Rendering 방식으로 클라이언트로 전송할 HTML 파일 구현

◦ templates/door_index.html

```
<!DOCTYPE html>
<html>
<head>
    <!-- 스마트폰 화면 크기에 맞춰 웹페이지를 맞춰주는 필수 태그 -->
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>LED</title>
</head>
<body>
    <h1>Smart door Control</h1>

    <!-- 파이썬에서 넘겨받은 현재 문의 상태(문자열) 출력 -->
    <h3>현재 문 상태: {{ door_status }}</h3>
    <hr>

    <!-- GET 방식 제어(웹 브라우저 제어용) -->
    <p>
        <a href="/get?action=OPEN"><button>문 열기</button></a>
        <a href="/get?action=CLOSE"><button>문 닫기</button></a>
    </p>

</body>
</html>
```

Jinja2가 사용된 부분

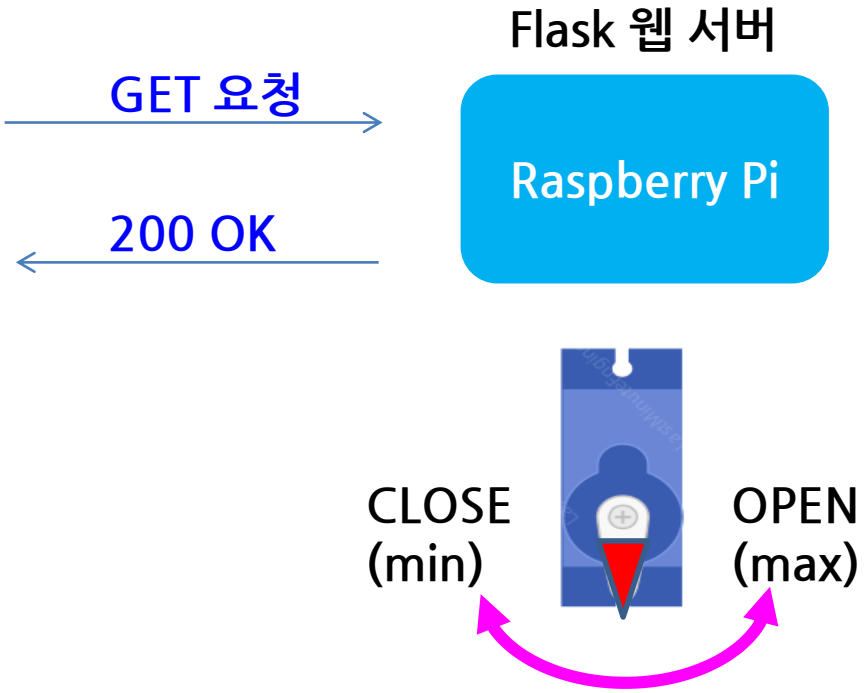
2. 스마트 도어락 - SSR 방식 실행

❑ SSR Server-Side Rendering 방식 테스트

- 실행하기

http://라즈베리파이 IP:5001/get?action=OPEN





3. 스마트 도어락 - CSR 방식

❑ CSR Client-Side Rendering 방식으로 라즈베리파이 서버 구현

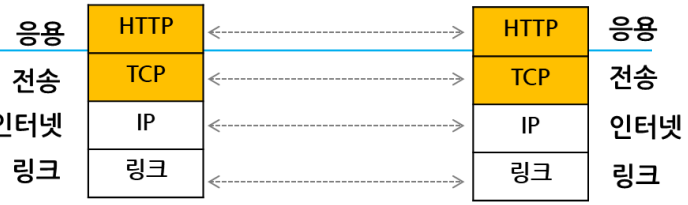
클라이언트 앱은 앱 인벤터를 이용해 제작합니다.

Flask 웹 서버는 SSR 방식의 GET 방식으로 웹 브라우저가 전송한
파라미터 데이터를 받아 서보모터를 제어해 보았습니다.

이젠 CSR 방식으로 앱에서 POST 방식으로 door의 action 데이터를 받아서
서보모터의 방향을 제어해 봅시다.

기존 door_app.py에 POST 방식을 추가로 구현해 주세요

3. 스마트 도어락 - CSR 방식



❑ CSR Client-Side Rendering 방식으로 스마트 도어락 구현하기

- **CSR 방식** : 브라우저(클라이언트)가 웹 화면을 직접 만드는 방식으로, 서버는 순수한 데이터(JSON 혹은 텍스트)만 전송하는 방식
- **POST 요청** : 모든 파라미터를 요청메시지 바디에 실어 요청하는 방식

클라이언트



http://라즈베리파이 IP주소:5001

POST 요청

요청메시지

"action=OPEN"

200 OK

응답메시지

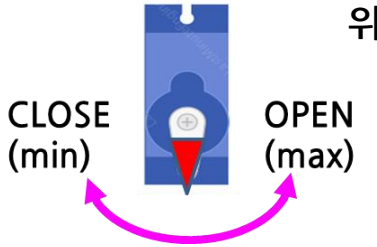
"DOOR_STATUS:{action}"

Flask 웹 서버

Raspberry Pi

예) 라즈베리파이
IP 주소: 10.2.20.162
PORT 번호: 5001

서보모터 방향제어를 위해서 PWM 사용



3. 스마트 도어락 - CSR 방식: 라즈베리파이 웹 서버 구현

❑ CSR Client-Side Rendering 방식을 처리하도록 라즈베리파이 웹 서버 구현

- 이름_7주차_door_app.py 에 POST 방식 처리 코드 추가 구현

```python
# 3. CSR방식. POST 요청시 처리 (앱 인벤터 제어용)
@app.route('/post', methods=['POST'], strict_slashes=False)
def post_door():
    action = request.form.get('action') # POST 요청메시지 body에서 action 값 읽기
    print("POST요청", action)

    if action in ['OPEN','CLOSE']: # OPEN 혹은 CLOSE인 경우
        control_door(action)

    # 앱 인벤터가 성공여부를 확인할 수 있도록 결과 문자열 반환
    return f"DOOR_STAUTS: {current_status}"

if __name__ == "__main__":
    # 앞선 실습과 충돌나지 않도록 포트는 5001번으로 수정
    app.run(host='0.0.0.0', port=5001)
```

3. 스마트 도어락 - CSR 방식: IoT 서비스 앱 제작

❑ 클라이언트 앱 제작 방법

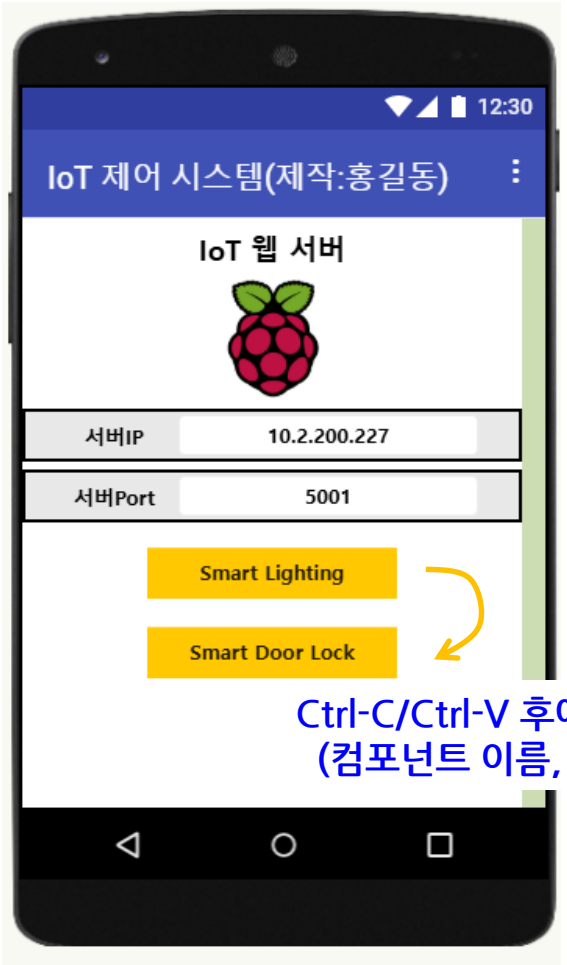
우리가 제작할 여러 클라이언트 앱은 큰 틀에서는 동일한 로직으로 작동합니다
다만 사용자가 서버로 전달하는 요청사항만이 다를 뿐입니다
매번 새롭게 제작하기 보다는 기존의 디자인과 코드를 재활용해서 제작합시다.



3. 스마트 도어락 - CSR 방식: IoT 서비스 앱 제작

❑ Screen1 (홈 화면) 디자인 추가하기

'IoT_System_학번' 프로젝트에 추가하여 제작



Screen1에 버튼 추가

	컴포넌트	이름	속성
①	버튼	버튼1_서보모터	배경색-주황, 글꼴굵게, 글꼴크기-15, 너비 -50% 모양-직사각형, 텍스트 - Smart Door Lock, 텍스트정렬- 가운데

- 스크린 3 추가하기

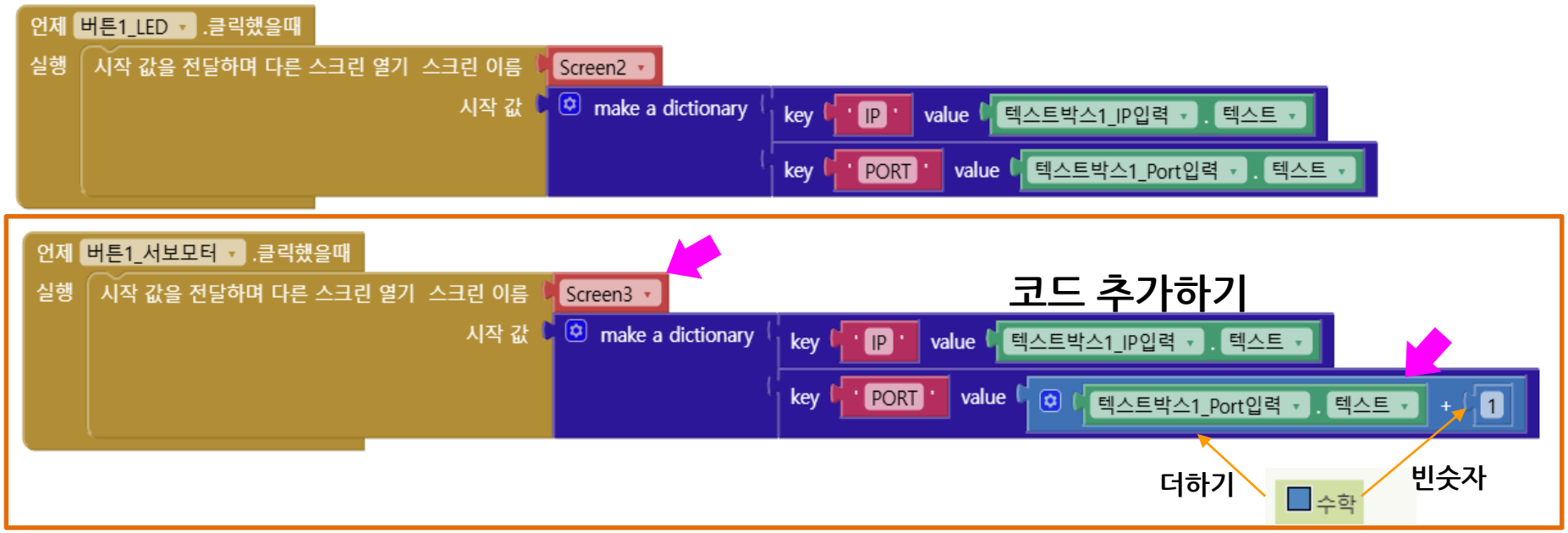




Ctrl-C/Ctrl-V 후에 필요한 부분만 변경
(컴포넌트 이름, 속성에서 텍스트)

3. 스마트 도어락 - CSR 방식: IoT 서비스 앱 제작

❑ Screen1 코드 추가하기

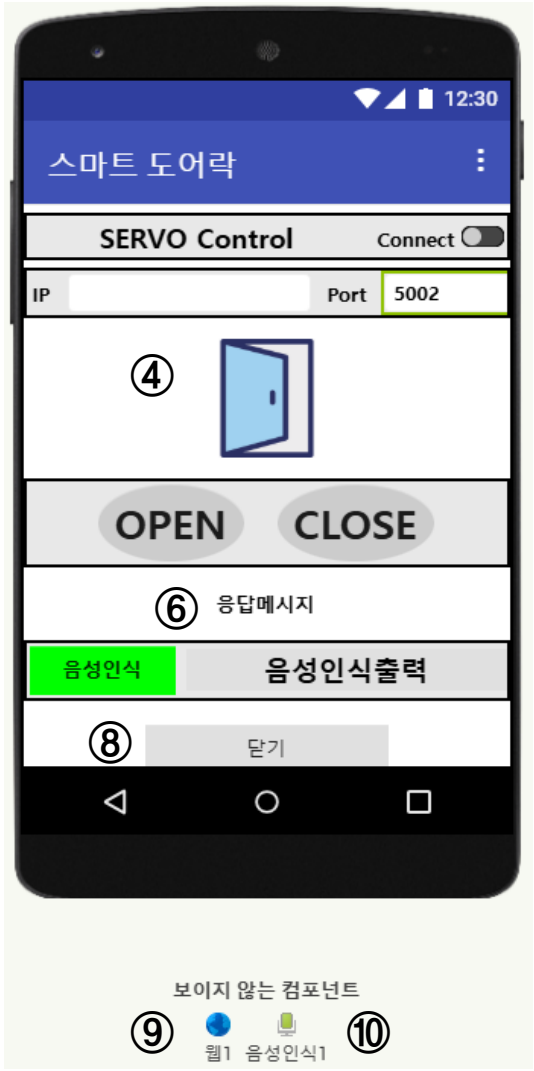


코드동작: Smart Door Lock 버튼을 클릭 했을 때 Screen3로 앱 화면이 전환됨
이때, 사용자가 텍스트박스에 입력한 IP주소와 Port번호(기존Port번호+1)을 시작 값으로 전달함

3. 스마트 도어락 - CSR 방식: IoT 서비스 앱 제작

❑ Screen3 디자인하기

Tip) Screen2에서 디자인을 재활용해 보세요

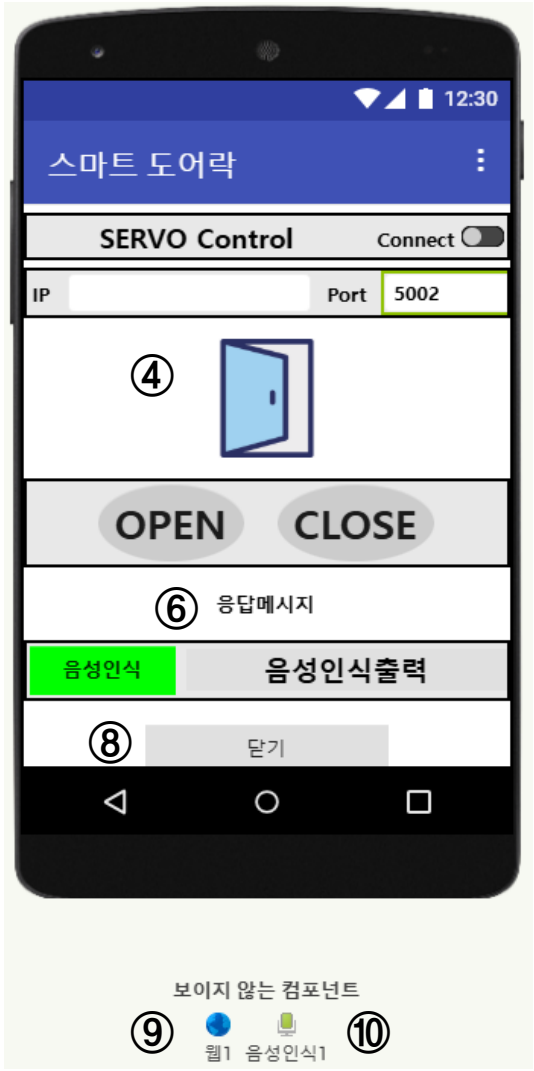


Screen3

	컴포넌트	이름	속성
①	Screen	Screen3	수평정렬-가운데, 스크롤가능여부- 체크 제목 - 스마트 도어락
②	수평배치	수평배치1	수평, 수직정렬-가운데, 너비-부모요소
	레이블	레이블1_title	글꼴굵게, 글꼴크기-20, 너비 - 부모요소 텍스트- 서보모터 제어, 텍스트정렬- 가운데
	스위치	스위치1	글꼴굵게, 글꼴크기-15, 텍스트 - Connect
③	수평배치	수평배치1	수평, 수직정렬-가운데, 너비-부모요소
	레이블	레이블1_IP	글꼴굵게, 글꼴크기-15, 너비-20% 텍스트- IP, 정렬-가운데
	텍스트박스	텍스트박스1_IP	글꼴굵게, 글꼴크기-15, 너비-30%, 정렬- 가운데
	레이블	레이블1_Port	글꼴굵게, 글꼴크기-15, 너비-20%, 텍스트- Port, 정렬-가운데
	텍스트박스	텍스트박스1_Port	글꼴굵게, 글꼴크기-15, 너비-30%, 정렬- 가운데, 숫자만
④	이미지	이미지1	높이-20%, 너비-20% 사진 - 도어락 이미지 (자유로)

3. 스마트 도어락 - CSR 방식: IoT 서비스 앱 제작

❑ Screen2 디자인하기(계속)



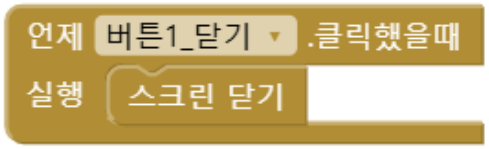
Screen3

	컴포넌트	이름	속성
⑤	수평배치	수평배치1	수평, 수직정렬-가운데, 너비-부모요소
	버튼	버튼1_Open	글꼴굵게, 글꼴크기-30, 모양 - 타원, 텍스트- OPEN, 정렬-가운데
	버튼	버튼1_Close	글꼴굵게, 글꼴크기-30, 모양 - 타원, 텍스트- CLOSE, 정렬-가운데
⑥	레이블	레이블1_응답메시지	글꼴굵게, 글꼴크기-15 텍스트 - 응답메시지, 텍스트정렬- 가운데
⑦	수평배치	수평배치1	수평, 수직정렬-가운데, 너비-부모요소
	버튼	버튼1_음성인식	배경색-초록, 글꼴굵게, 글꼴크기-15, 너비 - 30%, 모양-직사각형, 텍스트-음성인식, 텍스트정렬- 가운데
	레이블	레이블1_음성인식출력	배경색-밝은회색, 글꼴굵게, 글꼴크기-20, 너비-부모요소, 텍스트 - 음성인식출력, 정렬-가운데
⑧	버튼	버튼1_닫기	글꼴굵게, 글꼴크기-15, 너비 -50% 모양-직사각형, 텍스트 -닫기, 텍스트정렬- 가운데
⑧	웹	웹1	
⑩	음성인식	음성인식1	

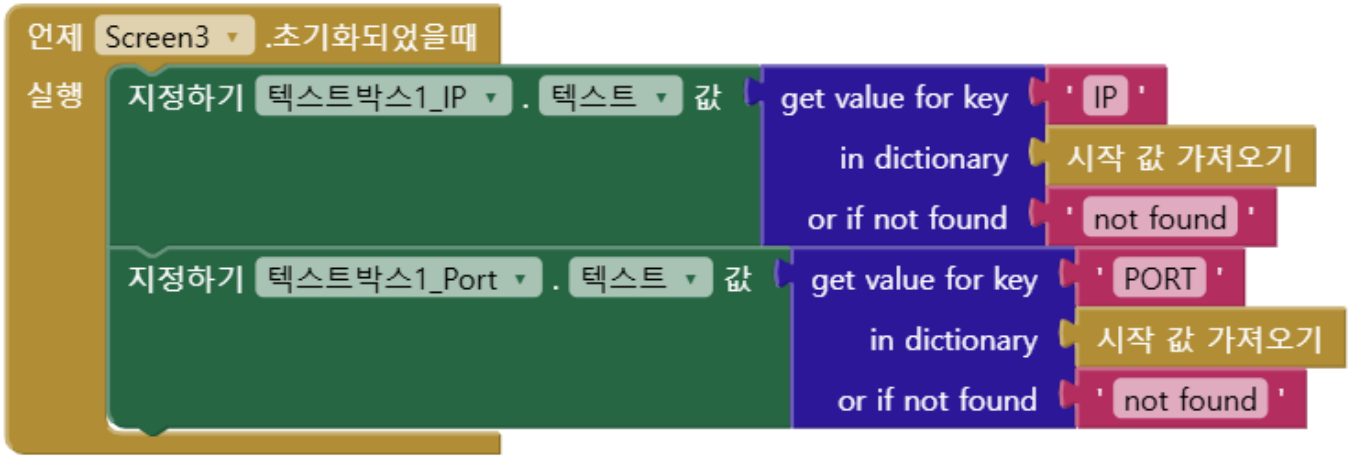
3. 스마트 도어락 - CSR 방식: IoT 서비스 앱 제작

❑ Screen3 코딩하기

Tip) Screen2의 블록의 코드를 재활용해 보세요



닫기 버튼을 클릭 했을 때, Screen3 화면을 닫고
Screen1(홈화면)으로 돌아가기



코드동작: 스크린3이 초기화 되었을 때(앱을 시작했을 때)
Screen1으로 부터 시작값(IP정보와 Port번호)을 가져와서 출력하기

3. 스마트 도어락 - CSR 방식: IoT 서비스 앱 제작

❑ Screen3 코딩하기(계속)

Tip) Screen2의 블록의 함수를 재활용해 보세요(매개변수 하나 삭제)

코드동작: Open, Close 버튼 클릭 시 POST 함수 호출



Form형식의 Body 작성 후 POST 전송

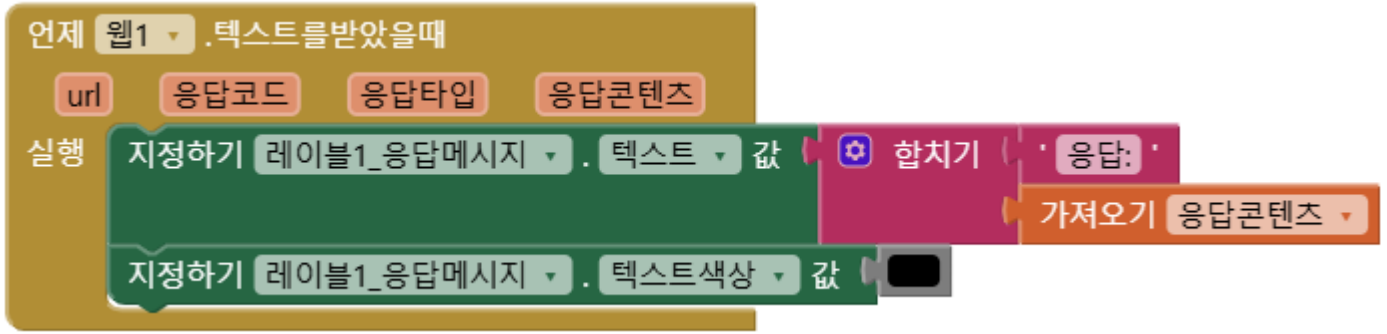
바디의 form형식: “action=OPEN”

3. 스마트 도어락 - CSR 방식: IoT 서비스 앱 제작

❑ Screen2 코딩하기(계속)

Tip) Screen2의 블록의 코드를 재활용해 보세요

코드동작: 서버의 응답메시지 수신 후 레이블에 출력



코드동작: 서버와의 연결에 실패하는 경우 오류 메시지 출력

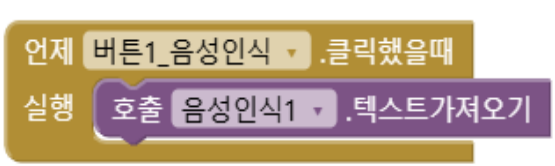


3. 스마트 도어락 - CSR 방식: IoT 서비스 앱 제작

❑ Screen3 코딩하기(계속)

Tip) Screen2의 블록의 코드를 재활용해 보세요(포함할 텍스트만 변경)

코드동작: 음성 명령을 통해 서보모터 방향 제어





3. 스마트 도어락 - CSR 방식 실행

❑ 스마트 도어락 실행하기



"서보모터 동작 확인하기"

POST 방식

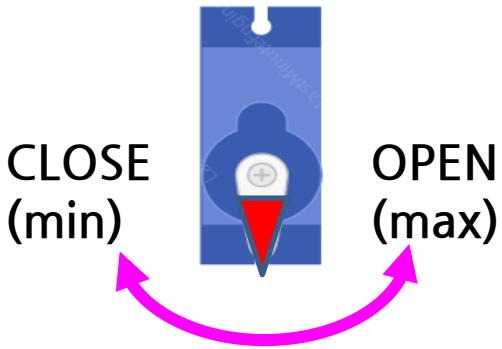
"action=OPEN"

200 OK

"DOOR_STATUS: {action}"

Flask

RaspberryPi



7주차 과제 제출 안내

❑ 7주차 실습과제

- 과제: 3교시 실습한 스마트 도어락 실습 파일을 제출해 주세요

- 제출파일: '이름_7주차_door_app.py', 'IoT_System_학번.aia'

** 실습 시 잘 안된 부분이 있다면 comment를 남겨 주세요*

IoT실습심화 중간고사 안내

- ❑ 시험일자 및 장소: 2026년 10월 21일(수) 09:30 ~ , 장소: C401
- ❑ 시험범위: 수업시간에 실습한 내용 위주 (강의자료 참조)
  *강의자료는 ctl.gtec.ac.kr 사이트에서 다운받으세요

주차	구분	내용	출제비율
1	IoT 개요 라즈베리파이 일반	• IoT 개요 • 라즈베리파이 일반 사항	대략 20%
2~3	LED 제어 RGB LED 제어	• LED 작동원리, PWM 원리이해 • RGB LED 회로구성 및 제어 코드(rgb.py) 이해	대략 20%
4	서보모터 제어	• 서보모터 방향제어 코드(servo.py) 이해 (스마트도어락과 중복출제)	대략 10%
5	Flask 웹 서버	• Flask 템플릿과 템플릿엔진 jinja2 개념	대략 10%
6	스마트 조명	• 스마트조명, 스마트도어락 구현 환경 및 동작절차 이해 • Flask 웹 서버 구현방식 이해(RGB_app.py, door_app.py) • Jinja2 변수 사용법 이해 (RGB_index.html. door_index.html) • 소자 제어하는 핵심 코드 이해	대략 40%
7	스마트 도어락		

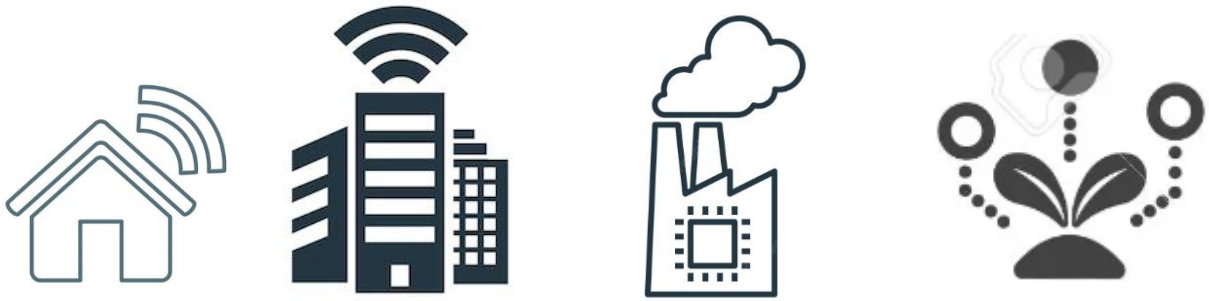
- ❑ 출제방식 : 객관식(단답형 포함)과 주관식 출제
- ❑ 주관식: 주1) RGB LED 제어 문제: RGB LED 회로연결 및 RGB LED 색상 제어 코드 (rgb.py 참조)
  주2) 스마트조명 문제: 스마트조명에 사용된 기술 및 핵심코드 이해 (코드 지문제시)
  주3) 스마트도어락 문제: 스마트도어락에 사용된 기술 및 핵심코드 이해 (코드 지문제시)
  주4) 용어 설명

1. IoT(Internet of Things)란?

◦ **IoT (사물인터넷) : 사물이 인터넷(네트워크)에 연결되고 지능화 되는 기술**

- 연결: 무선통신기술과 인터넷 환경을 통해 각종 사물을 연결하는 기술
- 지능화: 사물 스스로 정보를 분석하고, 학습하는 인공지능 기술

→ 센서가 부착된 사물을 인터넷(네트워크)에 연결하여 사람의 개입 없이 정보를 주고받는 환경과 기술



스마트홈 스마트시티 스마트팩토리 스마트팜

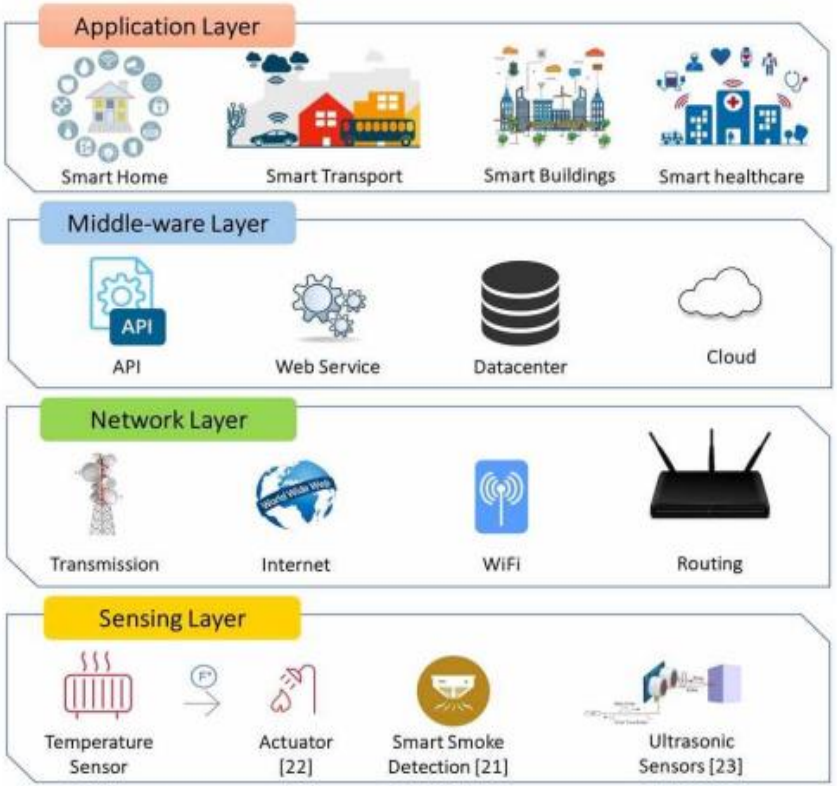
스마트란? → 사물이 인터넷(네트워크)에 연결되고, 지능화된다

◦ **IoT (사물인터넷) 의 핵심가치: 편의성과 효율성**

1. IoT(Internet of Things)란?

◦ IoT 3가지 주요 기술: 센싱기술 + 네트워킹기술 + 서비스 인터페이스 기술





서비스 ↑ 처리 ↑ 연결 ↑ 센싱

서비스 AI/분석

데이터 전송

센서 데이터 수집

(https://www.researchgate.net/figure/Layers-in-IoT-System_fig2_333909259)

◦ IoT 처리단계 : 센싱(Sensing) ➔ 연결(Networking) ➔ 처리(Processing) ➔ 서비스

1. 라즈베리파이

◦ **라즈베리파이는 작은 컴퓨터**

라즈베리파이(RaspberryPi)는 데비안 **리눅스 기반**의 라즈베리파이 OS를 운영체제로 사용하는 SBC(Single Board Computer)

- 오픈 소스 하드웨어로서 크기가 작고 저렴함
- 저비용, 초소형 컴퓨터로 인터넷검색 및 고화질비디오의 재생, 스프레드시트, 워드프로세싱, 게임등 데스크톱 컴퓨터에서 작업할 수 있다고 기대되는 모든 일을 할 수 있음
- 범용 입출력(GPIO) 핀을 통해 외부 전자장치와 연결하여 사용할 수 있음

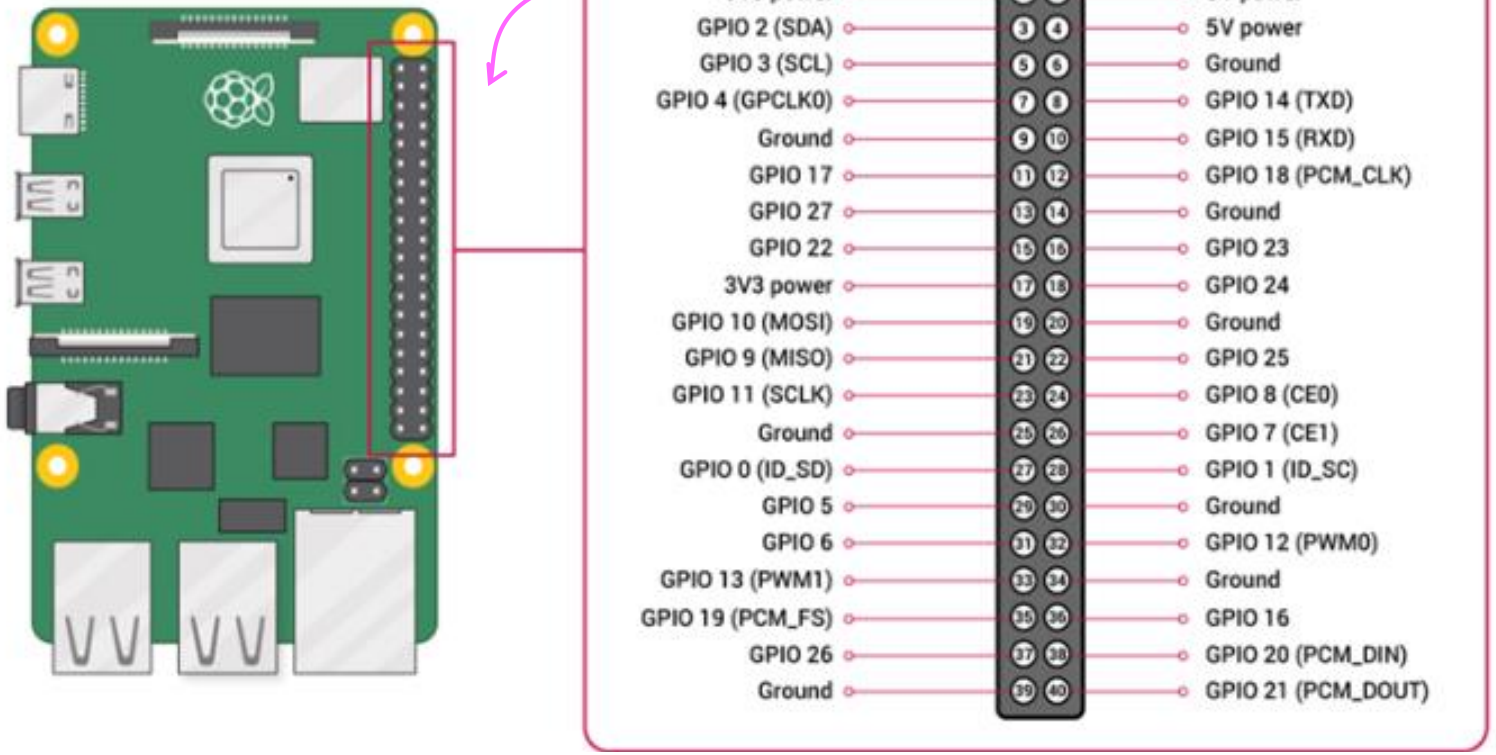


사물인터넷을 위한 소프트웨어를 프로그래밍할 수 있는 대표적인 오픈소스 하드웨어

1. 라즈베리파이

❑ GPIO (General Purpose Input / Output)

- 입/출력 신호를 제어할 수 있는 전용 포트(핀)
- GPIO 핀을 통해 LED, 모터, 부저등 액츄에이터를 제어
- GPIO 핀을 통해 센서등과 통신



출처: https://www.raspberrypi.com/documentation/computers/raspberry-pi.html#raspberry-pi-4-model-b

1. 라즈베리파이

❑ GPIO pin map 확인 (BCM mode)

- BCM(Broadcom) 모드

  라즈베리파이 칩(Broadcom SOC)에 직접 할당된 GPIO 핀 번호(실제 핀 번호)를 사용하겠다는 설정
  *GPIO 실습에 사용

  ※ Board 모드: 실제 보드 핀의 물리적인 번호를 사용

  - **GPIO pin map 확인하는 방법**

```
pi@raspberrypi:~ $ pinout
```

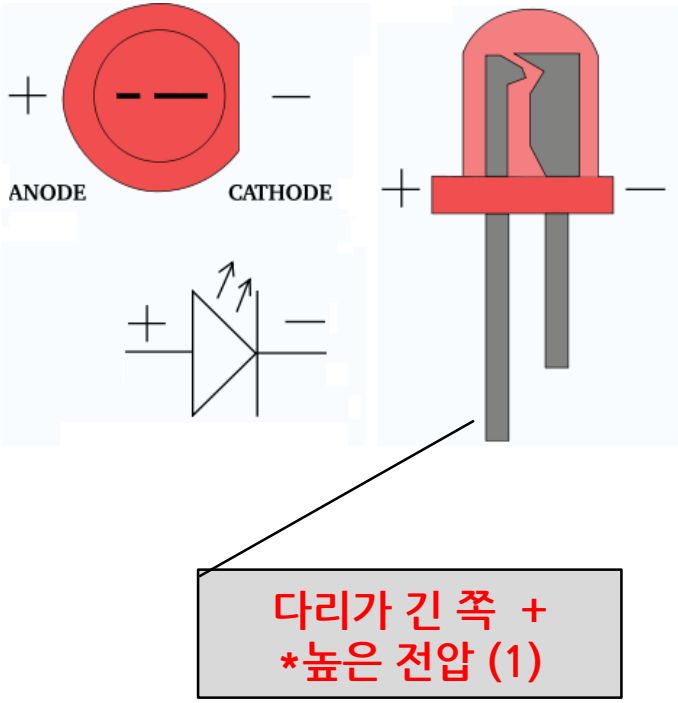
BCM BCM

```
   3V3  (1) (2)  5V
 GPIO2  (3) (4)  5V
 GPIO3  (5) (6)  GND
 GPIO4  (7) (8)  GPIO14
   GND  (9) (10) GPIO15
GPIO17 (11) (12) GPIO18
GPIO27 (13) (14) GND
GPIO22 (15) (16) GPIO23
   3V3 (17) (18) GPIO24
GPIO10 (19) (20) GND
 GPIO9 (21) (22) GPIO25
GPIO11 (23) (24) GPIO8
   GND (25) (26) GPIO7
 GPIO0 (27) (28) GPIO1
 GPIO5 (29) (30) GND
 GPIO6 (31) (32) GPIO12
GPIO13 (33) (34) GND
GPIO19 (35) (36) GPIO16
GPIO26 (37) (38) GPIO20
   GND (39) (40) GPIO21
```

2. 파이썬으로 LED 제어하기

❑ LED (Light-emitting diode ,발광다이오드)

- 순방향으로 전압을 가했을 때 발광하는 반도체 소자



* LED의 특징
  - 전류가 한방향으로만 흐르는 반도체 소자

1. LED는 극성이 있다 (긴 쪽, 짧은 쪽)

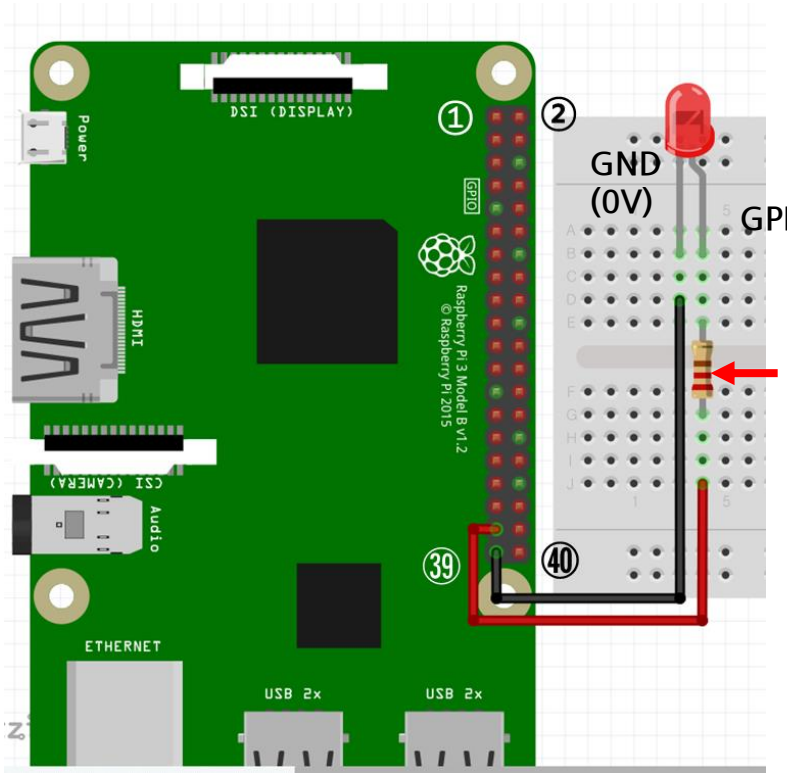
2. 긴 쪽에 높은 전압, 짧은 쪽에 낮은 전압이 걸렸을 때 전류가 흐르고 불이 켜짐
   *통상 짧은 쪽에는 GND(0V) 연결

3. LED로 과도한 전류가 흐르는 것을 막기위해 소자보호용으로 저항을 사용(330Ω)

2. 파이썬으로 LED 제어하기

❑ LED 회로연결 및 기본코딩 (blink)

- gpiozero 라이브러리는 파이썬용 하드웨어 제어 라이브러리로 객체지향제어로 쉽고, 직관적
- 기존의 RPi.GPIO 라이브러리 대체



저항용도:
과전류 방지
소자 보호용

```
from gpiozero import LED
import time

# GPIO 26번 핀에 연결된 LED 객체생성
led = LED(26)

print("LED blink test start!")

try:
    while True:
        print("LED ON")
        led.on()   # LED ON
        time.sleep(1)

        print("LED OFF")
        led.off()  # LED OFF
        time.sleep(1)
except KeyboardInterrupt:
    # Ctrl+C 누르면 핀을 초기화하고 종료하기
    print("LED blink test End!")
```

led.blink( )로 대체가능

* blink( )메소드는 백그라운드 스레드로 동작

3. PWM을 이용한 RGB LED 밝기 제어

❑ 라즈베리파이 RGB LED에 PWM을 사용하는 방법

- gezogpio RGBLED 클래스는 기본적으로 PWM 기능이 내장되어 동작
- 라즈베리파이의 디지털 출력을 쪼개는 **소프트웨어 PWM 기술을 활용**하여 구동

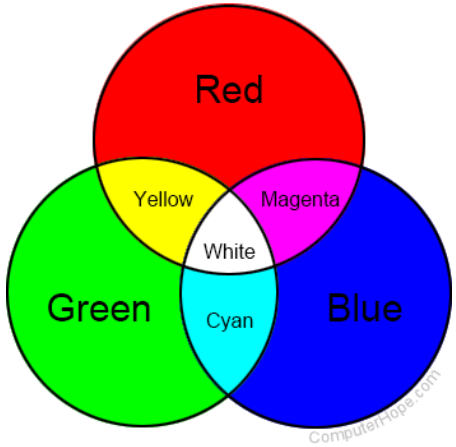
```
# RGB LED 객체 생성, 핀순서(r,g,b)
rgb = RGBLED(red=17, green=22, blue=27)
print(type(rgb._leds[0]))
```

```
<class 'gpiozero.output_devices.PWMLED'>
```

- 자동 듀티 사이클(Duty Cycle) 변환

rgb.color = (1.0, 0.0, 0.5) 입력 시
각 핀에 탑재된 softPWM 엔진에 다음과 같이 명령

1. Red 핀: PWM 듀티 사이클 100% 공급 (최대 밝기)
2. Green 핀: PWM 듀티 사이클 0% (완전히 끔)
3. Blue 핀: PWM 듀티 사이클 50%로 전압을 공급



3. PWM 이란?

"디지털 신호를 이용해서 아날로그처럼 제어해 봅시다"

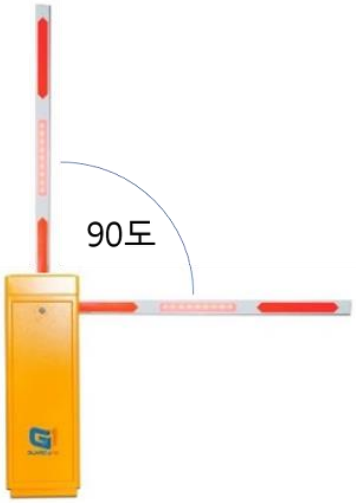
LED를 점 점 밝게…. 점 점 어둡게….
모터의 방향을 를 좀 더 왼쪽으로…. 좀 더 오른쪽으로….

PWM(Pulse Width Modulation: 펄스 폭 변조)은
디지털 신호를 이용해 아날로그 회로처럼 제어하는 방법입니다.

펄스의 폭을 조정하여 LED의 밝기와 서보모터의 방향을 제어에 활용



https://www.ilovepc.co.kr/news/

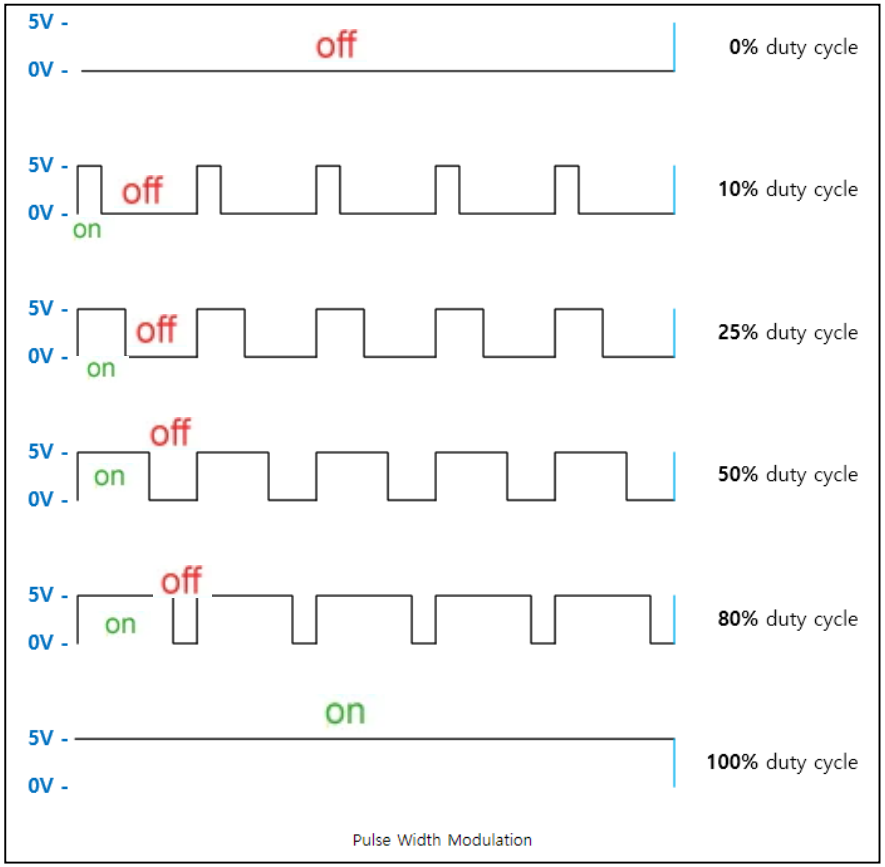


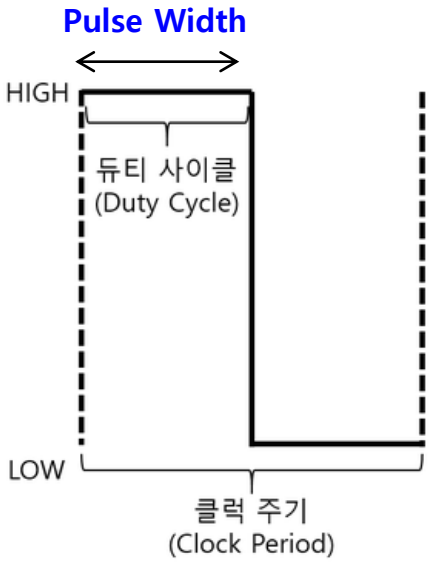
https://t1.daumcdn.net/cfile/tistory

3. PWM 동작원리

"좀 더 밝게 … 좀 더 어둡게…" 디지털로 아날로그를 제어해 보자
라즈베리파이와 함께 PWM을 사용하여 아날로그 출력을 다룹니다

❑ PWM(Pulse Width Modulation): 펄스 폭 변조로 디지털 출력으로 아날로그 회로를 제어하는 기법 → PWM 기능 이용 LED 밝기조절





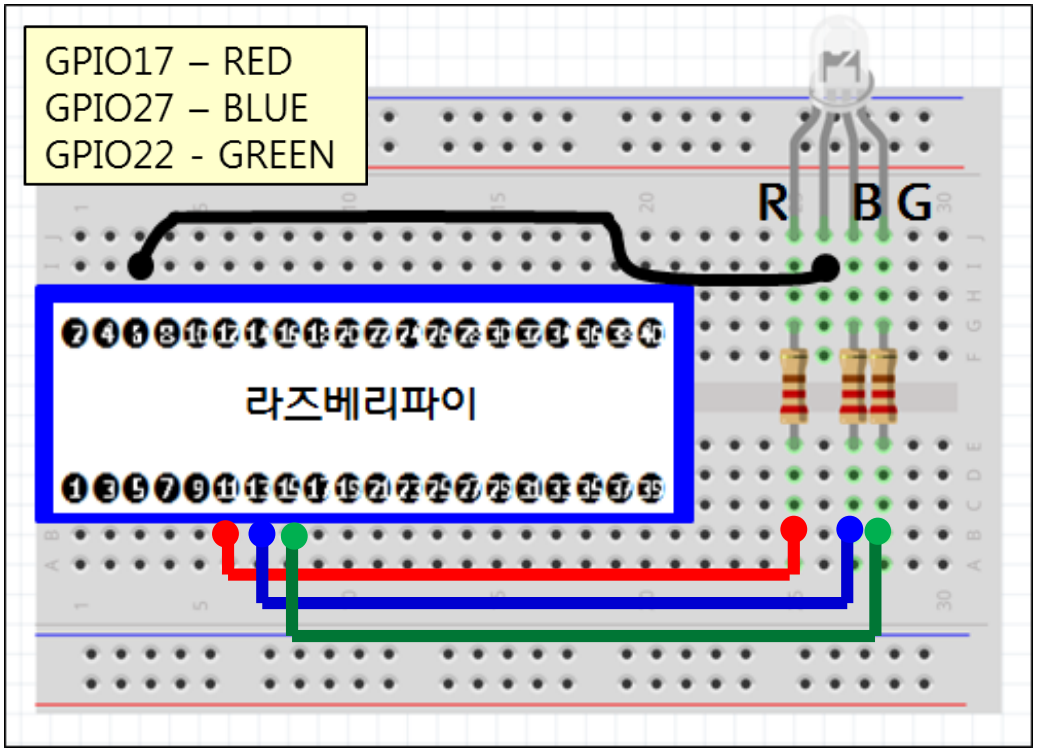
Duty cycle(듀티비) =

$$\frac{\text{Pulse Width}}{\text{주기(Period)}} \times 100$$

*주기(Period) = 하나의 사이클이 완성되는 시간

3. PWM을 이용한 RGB LED 밝기 제어

- RGB 동작회로 구성하기 (GPIO pin은 임의의 핀으로 변경됩니다)



3V3	(1)	(2)	5V
GPIO2	(3)	(4)	5V
GPIO3	(5)	(6)	GND
GPIO4	(7)	(8)	GPIO14
GND	(9)	(10)	GPIO15
GPIO17	(11)	(12)	GPIO18
GPIO27	(13)	(14)	GND
GPIO22	(15)	(16)	GPIO23
3V3	(17)	(18)	GPIO24
GPIO10	(19)	(20)	GND
GPIO9	(21)	(22)	GPIO25
GPIO11	(23)	(24)	GPIO8
GND	(25)	(26)	GPIO7
GPIO0	(27)	(28)	GPIO1
GPIO5	(29)	(30)	GND
GPIO6	(31)	(32)	GPIO12
GPIO13	(33)	(34)	GND
GPIO19	(35)	(36)	GPIO16
GPIO26	(37)	(38)	GPIO20
GND	(39)	(40)	GPIO21

3. PWM을 이용한 RGB LED 밝기 제어

❑ RGB 색상 제어 코드

rgb.py코드

```
# RGBLED 기본제어 방법:blink동작
from gpiozero import RGBLED
import time

# RGBLED pin 지정후 객체 생성
rgb = RGBLED(red=17, green=22, blue=27)

print("RGB LED 순서대로 blink하기")
try:
    while True:
        print("RED ON")
        rgb.color = (1, 0, 0) # (r, g, b)
        time.sleep(1)
        print("GREEN ON")
        rgb.color = (0, 1, 0)
        time.sleep(1)
        print("BLUE ON")
        rgb.color = (0, 0, 1)
        time.sleep(1)
except KeyboardInterrupt:
    pass
finally:
    # 예외처리 종료시 모든 리소스 cleanup 하기
    rgb.close()
    print("Program End")
```

1. gpiozero 라이브러리의 RGBLED 클래스이용

- RGBLED 모듈을 쉽게 제어할 수 있도록 만든 전용 도구
- 핵심 제어 수단: color 속성(Property)
- 기본적으로 softPWM으로 동작 전류의 양을 미세 조정하여 밝기조절

2. rgb.color = (값) 형식,

*** color는 속성(property)**

함수로 호출하는 것이 아니라, 일반 변수처럼 값을 직접 읽거나 바꿀 수 있도록 설계

- 데이터 형태: 3개의 소수점을 가진 튜플 형태 (Red, Green, Blue)
- 값의 범위: 각 색상 채널의 범위는 0.0(완전히 꺼짐)부터 1.0(최대 밝기)까지

4. 서보모터

❑ 서보모터(Servo Motor)

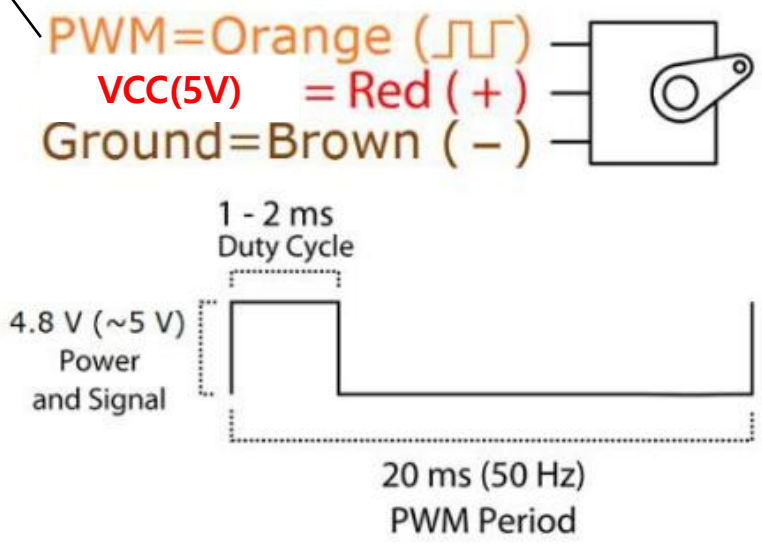
- 명령에 따라 정확한 위치와 속도를 제어할 수 있는 모터
- 응용: 자동화 생산 시스템, 로봇, 가전제품, CCTV 카메라, 캠코더, 프린터등 사용
- 종류: 표준 서보모터 (회전각도 0° ~ 180°)

- 서보모터에는 3개의 단자가 나와 있음



GPIO 포트에 연결

GPIO 포트에 연결해서 PWM 제어



[서보모터 규격]

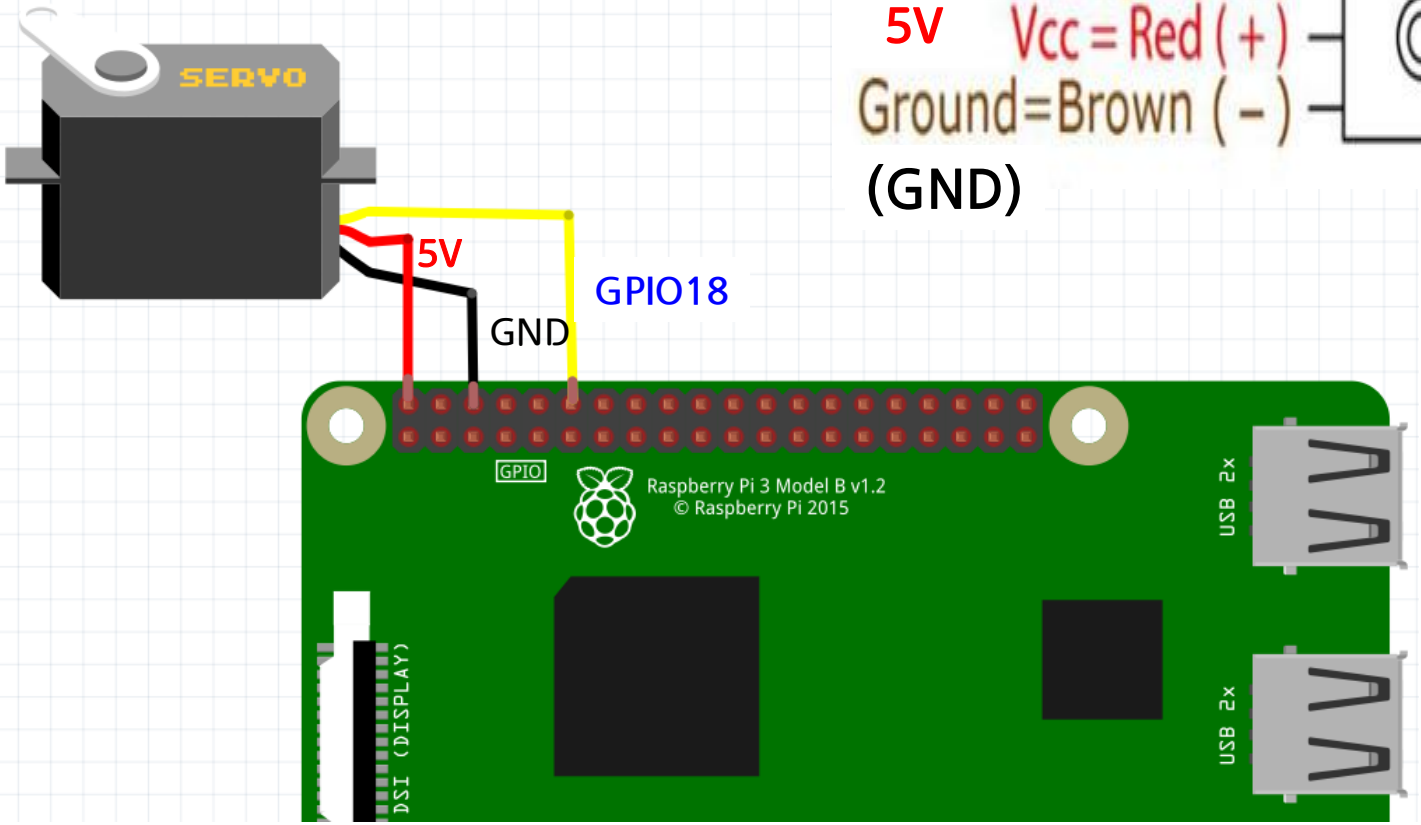
4. PWM을 이용한 서보모터 기본 제어

3V3	(1)	(2)	5V
GPIO2	(3)	(4)	5V
GPIO3	(5)	(6)	GND
GPIO4	(7)	(8)	GPIO14
GND	(9)	(10)	GPIO15
GPIO17	(11)	(12)	GPIO18
GPIO27	(13)	(14)	GND

❑ 서보모터 제어 - 회로 연결하기

- 서보모터의 GPIO는 18번 사용





4. PWM을 이용한 서보모터 기본 제어

❑ 보정값을 적용해 서보모터의 방향을 제어하기

* servo.py 코드

```
# PWM으로 서보모터 방향 제어하기
from gpiozero import Servo
import time

# SG90 서보모터 객체 생성(GPIO18번), PWM 조정값 적용
servo = Servo(18, min_pulse_width=0.5/1000, max_pulse_width=2.5/1000)

print("서보모터 위치제어 실습(전선을 정면으로 보기)")
print('='*45)
print("1:left(0도), c:center(90도), r:right(180도)")
print("프로그램 종료는 'q'입력")
print('='*45)
```

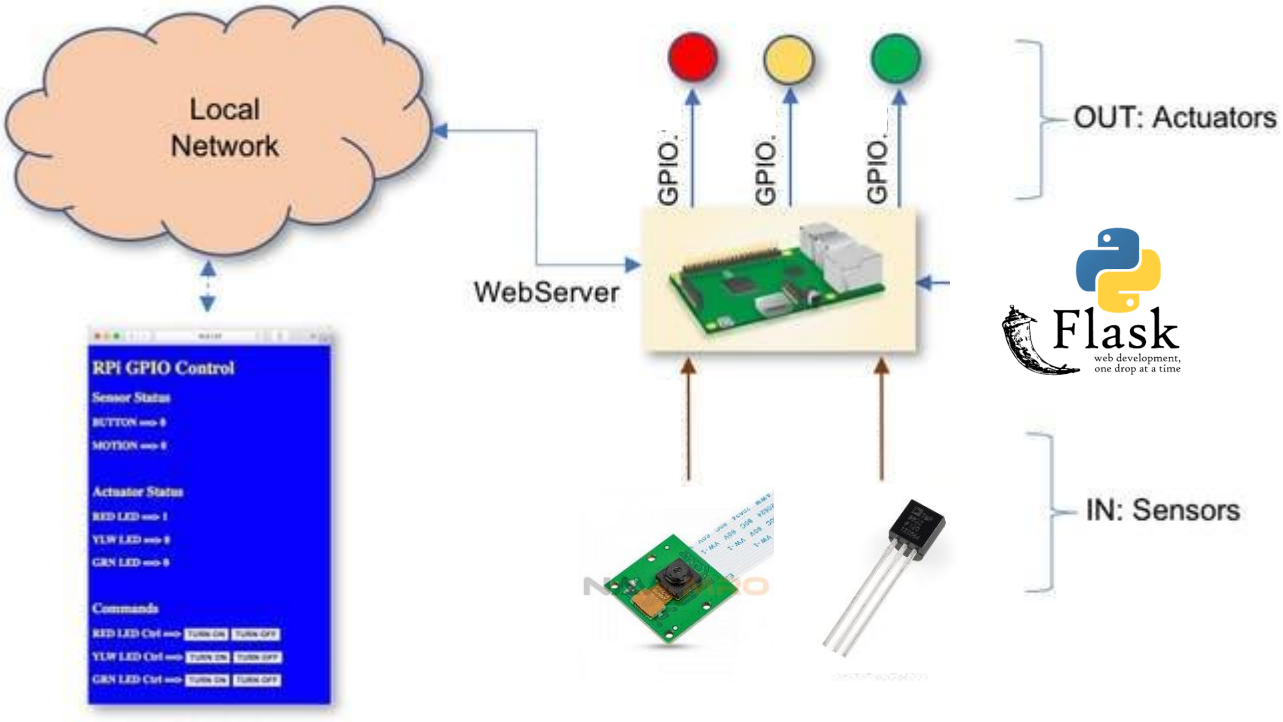
```
try:
    while True:
        # 사용자로 위치 입력받기
        command = input("원하는 위치를 선택하세요(l/d/r/q)")
        if command == 'l':
            print("Left 위치로 이동 (value: -1.0)")
            servo.min()
            #servo.value = -1.0   #동일하게 작동
        elif command == 'c':
            print("Center 위치로 이동 (value: 0.0)")
            servo.mid()
            #servo.value = 0.0    # 동일하게 작동
        elif command == 'r':
            print("Right 위치로 이동 (value: 1.0)")
            servo.max()
            #servo.value = 1.0     # 동일하게 작동
        elif command == 'q':
            print("프로그램을 종료합니다")
            break
        else:
            print("잘못된 입력입니다")
        # 모터가 해당위치까지 물리적으로 회전할 최소한의 시간 주기
        time.sleep(0.5)
        servo.detach()  # 이동 후 펄스신호를 끊어 떨림(jitter) 방지
except KeyboardInterrupt:
    pass
finally:
    servo.close()
```

모터를 원하는 방향으로 이동 후 detach()를 이용해 PWM 펄스 신호를 끊어 떨림(jitter)을 방지하기

5. 라즈베리파이 Flask 웹 서버

■ 라즈베리파이 Flask 웹 서버

- 라즈베리파이에서 파이썬 Flask를 사용해 가볍고 강력한 웹 서버 구축
- Flask는 파이썬 웹 프레임워크로 라즈베리파이의 하드웨어(GPIO) 제어나 센서 데이터 모니터링용 웹 페이지를 만들 때 자주 사용



그림참조: https://medium.com/data-science/

5. 라즈베리파이 Flask 웹 서버

■ Flask 기초

◦ **Flask는 파이썬 웹 프레임워크**
- Flask는 파이썬을 기반으로 한 **마이크로 웹 프레임워크**
- django는 파이썬을 기반으로 한 풀 웹 프레임워크



◦ **웹 프레임워크(Framework: 틀, 도구상자)란?**
- 웹 애플리케이션 개발을 쉽고 빠르게 할 수 있도록 미리 만들어진 구조와 도구를 제공하는 소프트웨어

◦ Flask는 기본적으로 HTTP 프로토콜을 사용하며, HTTP는 TCP를 기반으로 동작

◦ **라즈베리파이에 Flask 설치를 설치하기**

pip install flask

설치 에러 발생시)

```
pi@raspberrypi:~ $ pip install flask
error: externally-managed-environment
```

```
pi@raspberrypi:~ $ sudo rm /usr/lib/python3.13/EXTERNALLY-MANAGED
pi@raspberrypi:~ $ pip install flask
```

5. 라즈베리파이 Flask 웹 서버

■ 라즈베리파이 Flask 웹 서버 기본 코딩

◦ Flask 애플리케이션을 생성하기 위한 기본 코드

flask_demo.py

```
# Flask 웹 프레임워크 기본 사용법
from flask import Flask

# 1. Flask 웹 서버 객체 생성
app = Flask(__name__)

# 2. 라우팅 설정:어떤 주소(경로)로 요청이 있을시 어떤 함수가 실행될지 지정
# 사용자가 루트('/') 홈페이지에 접속 시 실행할 함수 지정
@app.route('/')
def home():
    return "<h1>Welcome Raspberry Web Server!</h1>"

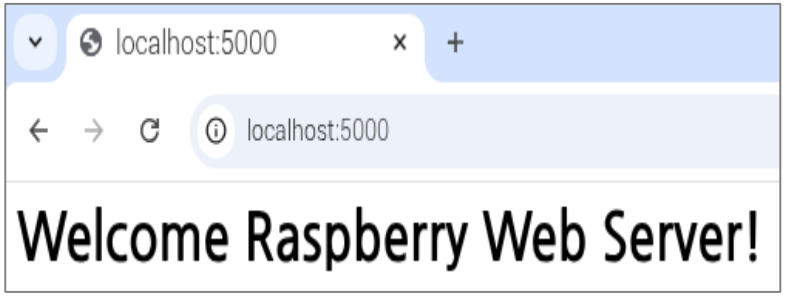
# 3. 서버실행
# default는 IP주소는 로컬('localhost' 또는 '127.0.0.1'), port번호는 5000
if __name__ == "__main__":
    app.run(host='localhost', port=5000)
```

5. 라즈베리파이 Flask 웹 서버

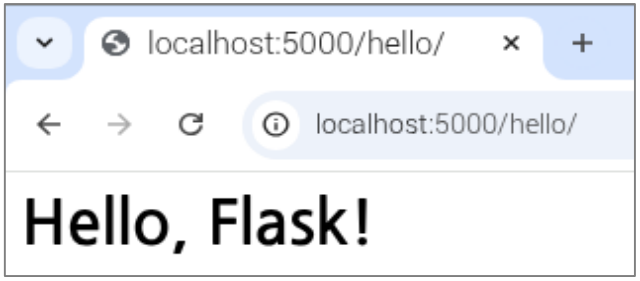
■ Flask 기초 : 라우팅

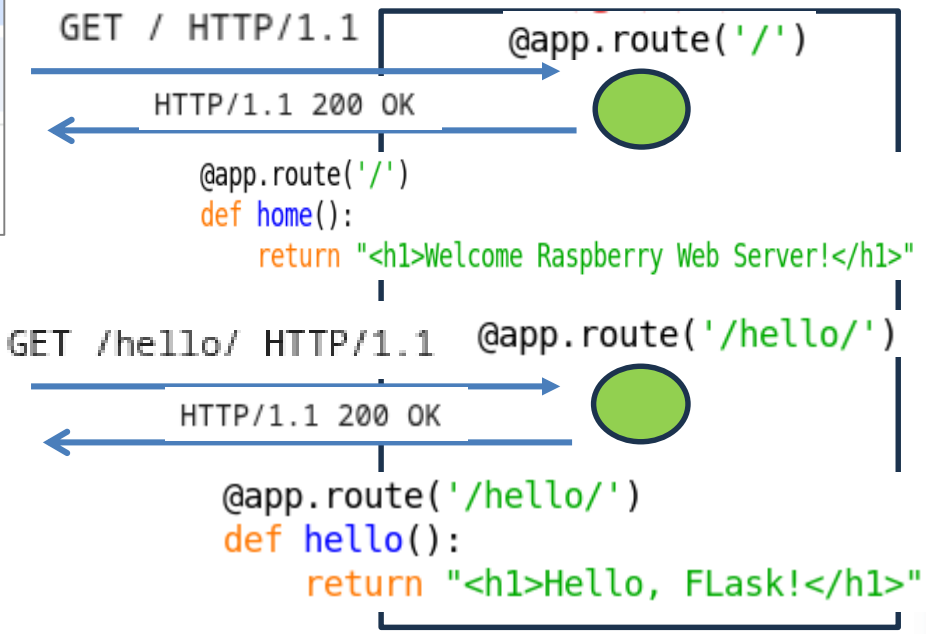
◦ Flask 라우팅: 특정 URL 요청이 들어왔을 때 해당 함수가 실행되도록 URL과 실행 함수를 지정하는(매핑하는) 역할

http://localhost:5001/



http://localhost:5000/hello

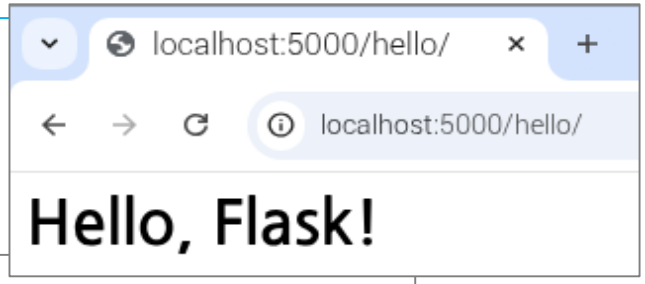




5. 라즈베리파이 Flask 웹 서버

■ Flask 기초 : 라우팅 추가하기

- http://localhost:5000/hello 경로 추가하기




```
# Flask 웹 프레임워크 기본 사용법
from flask import Flask

# 1. Flask 웹 서버 객체 생성
app = Flask(__name__)

# 2. 라우팅 설정:어떤 주소(경로)로 요청이 있을시 어떤 함수가 실행될지 지정
# 사용자가 루트('/') 홈페이지에 접속 시 실행할 함수 지정
@app.route('/')
def home():
    return "<h1>Welcome Raspberry Web Server!</h1>"

# 4. 경로추가: 사용자가 주소('/hello')로 요청시 라우팅 경로 추가하기
@app.route('/hello/')
def hello():
    return "<h1>Hello, Flask!</h1>"

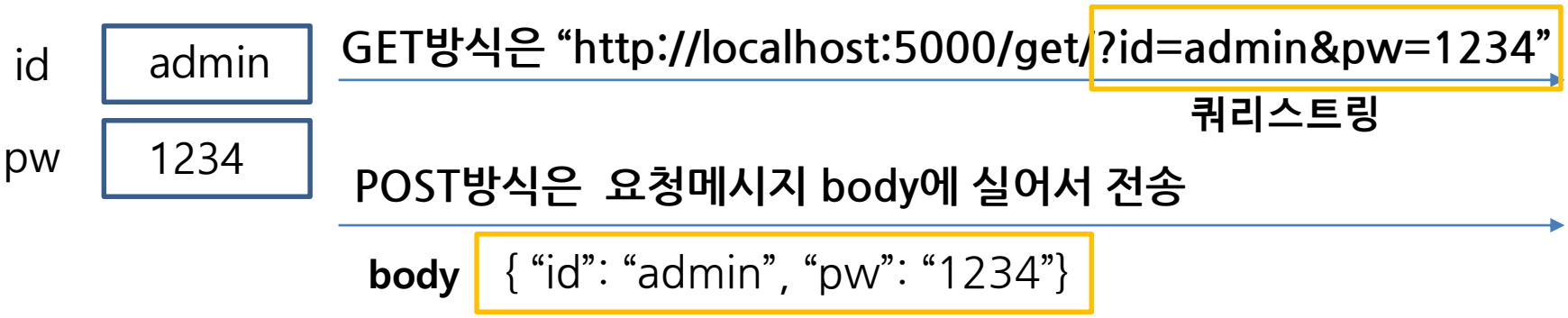
# 3. 서버실행
# default는 IP주소는 로컬('localhost' 또는 '127.0.0.1'), port번호는 5000
if __name__ == "__main__":
    app.run(host='localhost', port=5000)
```

5. 라즈베리파이 Flask 웹 서버

■ HTTP 요청 메소드 GET 방식과 POST 방식 비교

- GET 방식 요청과 POST 방식 요청의 차이

메소드	특징
GET	- 목적) 리소스를 얻어 오기 위한 요청 - 데이터 전달방식) 바디 사용 없이 URL에 쿼리 스트링으로 파라미터 데이터 전달
POST	- 목적) 새로운 리소스를 생성하기 위한 요청 - 데이터 전달방식) 바디를 사용해 파라미터 데이터 전달



5. 라즈베리파이 Flask 웹 서버

■ Flask 템플릿(template)과 Jinja2

- Flask 템플릿(template)과 Jinja2 사용하는 이유

1) 템플릿으로 파이썬 로직 코드와 HTML 화면 디자인을 서로 분리하여 관리

- 파이썬 파일에는 데이터 처리나 하드웨어 제어 로직만 남기고, HTML 파일은 별도 관리

2) 템플릿 엔진을 통해 동적 웹페이지 제공

- 정적 웹페이지와 동적 웹페이지의 차이는
  "누구에게나 항상 똑같은 화면을 보여주는가, 아니면 상황에 따라 다른 화면을 보여주는가“

- 정적 웹 페이지: 서버에 미리 저장된 HTML 파일을 그대로 사용자에게 보여주는 페이지
  (항상 똑같은 화면 제공)
- 동적 웹 페이지: 사용자의 요청, 시간, DB의 상태에 따라 실시간으로 HTML을 만들어 제공
  (접속자, 시간, 입력 값에 따라 화면의 내용이 바뀜)
  HTML + data처리 (파이썬 코드) 결합 → 동적 HTML

5. 라즈베리파이 Flask 웹 서버

■ Flask 템플릿(template)와 Jinja2 사용법

- 템플릿(Template) : 사용자에게 보여줄 HTML 틀
- Jinja2: HTML 코드안에서 파이썬 문법(변수, 반복문, 조건문)을 결합해서 동적 HTML문서를 만들어 주는 **Flask 템플릿 엔진** (https://jinja.palletsprojects.com)

→ 동적으로 페이지를 생성하기 위함

- 프로젝트 구조

```
🗀 프로젝트 폴더/
│  ├── app.py (Flask 서버 코드)
└── 🗀 templates/
        └── index.html (Jinja2를 사용할 HTML 파일)
```

※ 템플릿 엔진을 사용하기 위해서는 templates 폴더 밑에 HTML 파일이 존재해야 함

5. 라즈베리파이 Flask 웹 서버

■ Flask 템플릿엔진 Jinja2 사용법 - 변수사용법

◦ **실습에 사용된 flask_demo.py 코드를 Jinja2 사용해서 동적 html을 작성해 봅시다**

- Jinja2는 템플릿 엔진 : HTML 문서안에서 파이썬 코드를 결합해서 동적 HTML 문서를 만드는 라이브러리

- **Jinja2 변수 사용하기**
  {{ 변수명 }} **# HTML에 변수 출력하기**

Flask/
├── flask_template.py (Flask 서버 코드)
└── templates/
    └── hello.html (Jinja를 사용할 HTML 파일)

파일명: templates/**hello.html**

```
<!DOCTYPE html>
<html>
    <body>
        <h1>Hello, {{ name }}님! <br><br>
         Flask 템플릿 엔진 jinja2를 사용하여 변수를 반환합니다</h1>
    </body>
</html>
```

hello.html로 templates 밑에 저장

5. 라즈베리파이 Flask 웹 서버

■ Flask 템플릿엔진 Jinja2 사용법 - 제어문 사용법

- Jinja2 제어문 사용하기
  {% 제어문 %} # HTML에 제어문 사용
  {% end제어문 %} # 제어문 종료

◦ templates/fruits.html

```
<!DOCTYPE html>
<html>
    <body>
        <! --- 변수는 중괄호 두개 사용 --->
        <h1>안녕하세요, {{ name }}님!</h1>
        <hr>
        <h3>오늘의 과일 바구니</h3>
        <! --- 제어문은 중괄호와 퍼센트(%) 사용, end로 닫기 --->
        <ul>
            {% for item in items %}
                <li>{{ item }}</li>
            <!--- for ~ else는 리스트가 비어있거나 없는 경우 실행 --->
            {% else %}
                <li>장바구니가 비어 있습니다</li>
            {% endfor %}
        </ul>
    </body>
</html>
```

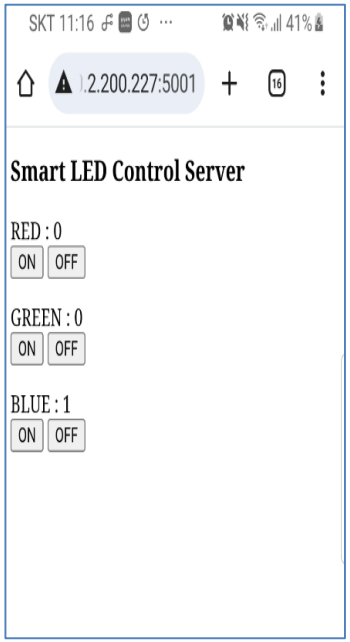
Flask/
├── flask_template.py (Flask 서버 코드)
└── templates/ (폴더 만들기)
    └── fruits.html (Jinja를 사용할 HTML 파일)

6. 스마트 조명 - 동작 환경

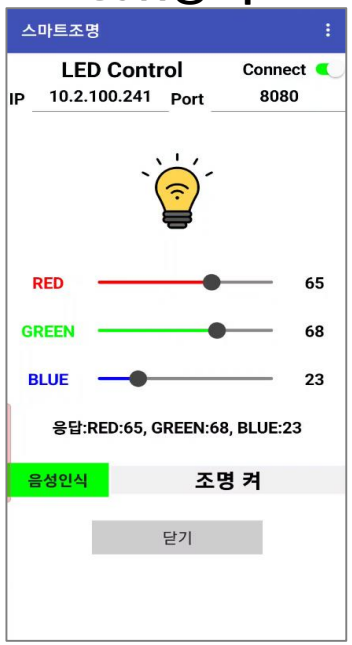
❑ 스마트 조명 프로토콜 스택

"스마트 조명을
2가지 방식으로 구현"

SSR방식



CSR방식



TCP/IP 프로토콜 스택







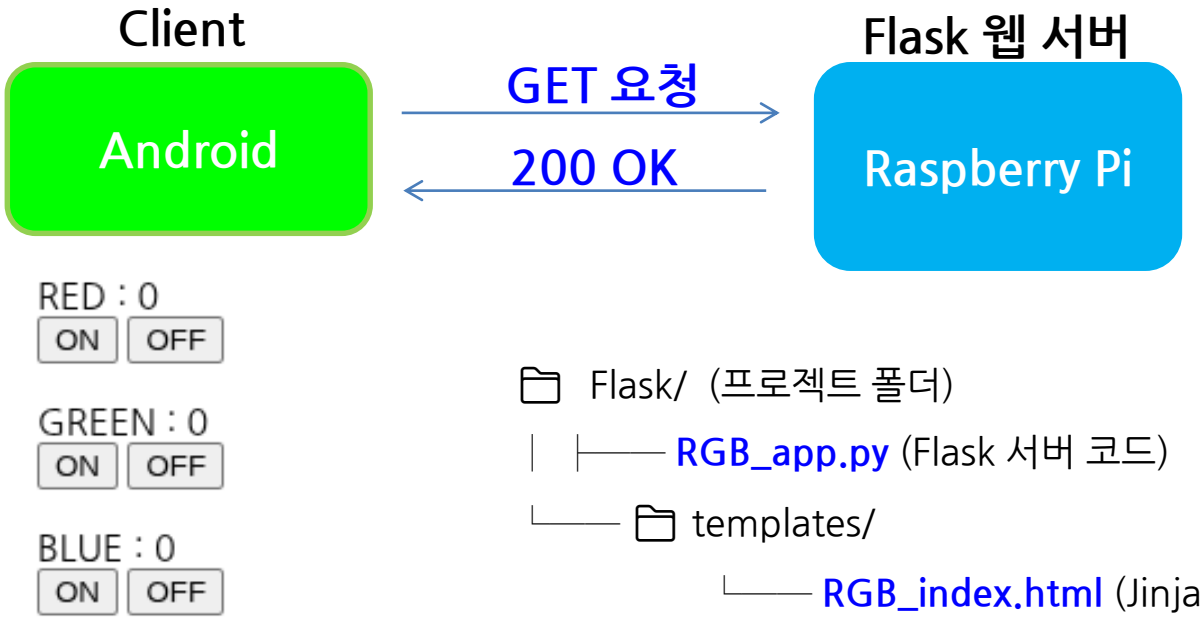
6. 스마트 조명 - 1) SSR 방식

❑ 실습1) SSR Server-Side Rendering 방식으로 라즈베리파이 서버 구현

[동작 개요]

- SSR 방식 : 서버가 데이터가 반영된 완성된 HTML 페이지를 만들어서 브라우저에 전달하는 방식
- GET 방식 사용 : 클라이언트가 파라미터를 URL 쿼리스트링으로 보내 요청하는 방식

http://라즈베리파이 IP:5000/get?color=RED&state=1



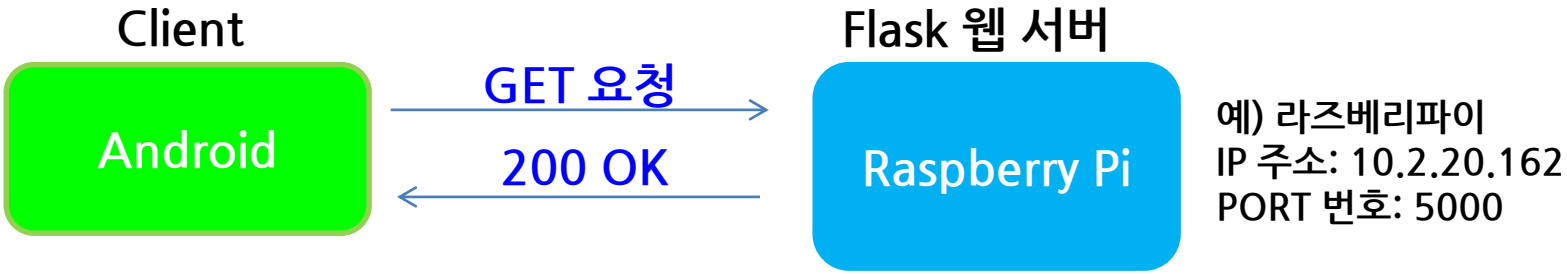
Flask/ (프로젝트 폴더)
├── RGB_app.py (Flask 서버 코드)
└── templates/
    └── RGB_index.html (Jinja2를 사용할 HTML 파일)

6. 스마트 조명 - 1) SSR 방식

❑ SSR Server-Side Rendering 방식으로 라즈베리파이 서버 구현

[동작 절차]

② http://라즈베리파이 IP:5000/get?color=RED&state=1



① RED: ON OFF
GREEN: ON OFF
BLUE: ON OFF

RGB_state_dict = {'RED': 0, 'GREEN':0, 'BLUE':0}

③ query string paring

{'color': 'RED', 'state': '1'}

④
```
color = request.args.get('color')
state = requests.args.get('state')
```

⑤
```
# 클라이언트 정보를 받아 RGB 색상과
제어정보 업데이트
    def update_RGB( ):
```

6. 스마트 조명 - 1) SSR 방식: 웹 서버 구현

❑ SSR Server-Side Rendering 방식으로 라즈베리파이 웹 서버 구현 - 서버 코드

◦ RGB_app.py

```python
# 스마트조명 웹 서버 프로그램
from flask import Flask, request, render_template
from gpiozero import RGBLED

# 웹 서버 객체 생성
app = Flask(__name__)

# RGBLED pin설정 및 객체 생성
rgb = RGBLED(red=17, green=22, blue=27)

# RGB 상태 DB (딕셔너리)
RGB_state_dict = {'RED': 0, 'GREEN': 0, 'BLUE':0}

# RGBLED 제어 핵심로직
def update_RGB():
    r_val = float(RGB_state_dict['RED'])
    g_val = float(RGB_state_dict['GREEN'])
    b_val = float(RGB_state_dict['BLUE'])
    rgb.color = (r_val, g_val, b_val)

update_RGB()  # RGBLED 초기화
```

RGB 핵심로직(PWM)

* strict_slashes=False는 URL 끝에 붙는 슬래시(/)를 엄격하게 구분하지 않겠다는 설정
(URL 끝에 슬래시가 있든 없든 두 경우 모두 정상적으로 같은 페이지(라우트)로 연결)

```python
# 루트로 접속 시 처리
@app.route('/', strict_slashes=False)
def index():
    return render_template('RGB_index.html', RGB_states=RGB_state_dict)

# SSR방식. GET 요청에 대한 처리 (웹 브라우저용)
@app.route('/get', methods=['GET'], strict_slashes=False)
def get_RGB():
    # URL 쿼리에서 key로 value 저장하기
    color = request.args.get('color')
    state = request.args.get('state')

    RGB_state_dict[color] = int(state)
    update_RGB()   # RGBLED 상태 업데이트

    return render_template('RGB_index.html', RGB_states=RGB_state_dict)

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000)
```

없어도 됨(default)

템플릿엔진으로 넘겨주는 값

'0.0.0.0'은 "모든 IP 주소"를 나타내는 특별한 예약 주소

6. 스마트 조명 - 1) SSR 방식: HTML 구현

❑ SSR Server-Side Rendering 방식으로 라즈베리파이 웹 서버 구현 - HTML 코드

- templates/RGB_index.html
  - 메인 코드에서 RGB_states_dict를 RGB_states로 전달 받음

```
return render_template('RGB_index.html', RGB_states=RGB_state_dict)
```

```
# 상태 저장 딕셔너리
RGB_state_dict = {'RED': 0, 'GREEN': 0, 'BLUE': 0}
```

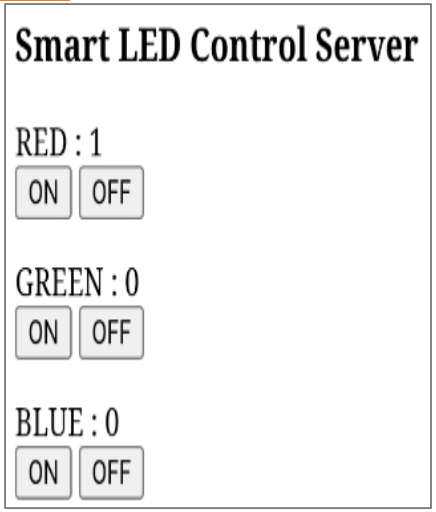
```
<!DOCTYPE html>
<html>
<head>
    <!-- 스마트폰 화면 크기에 맞춰 웹페이지를 맞춰주는 필수 태그 -->
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>LED</title>
</head>
<body>
    <h3>Smart LED Control Server</h3>

    <!-- Jinja2 반복문 시작 -->
    {% for color, state in RGB_states.items() %}
    <p>
        <!-- 딕셔너리의 key(색상)과 value(상태)를 받아 화면에 출력 -->
        {{ color }} : {{ state }}
        <br>
        <!-- 버튼 클릭시 주소창에 파라미터를 담아 서버로 전송 -->
        <a href="/get?color={{ color }}&state=1"><button>ON</button></a>
        <a href="/get?color={{ color }}&state=0"><button>OFF</button></a>
    </p>
    {% endfor %}
    <!-- jinja2 반복문 끝 -->

</body>
</html>
```

파이썬 딕셔너리(RGB_states)에 저장된
모든 key(키)와 value(값)의 쌍을 하나씩 꺼내어
반복문(for문)을 돌리겠다는 의미

Jinja2가 사용된 부분

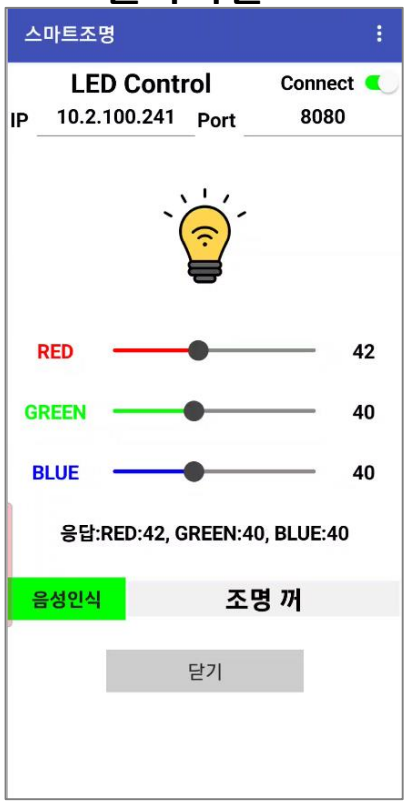


6. 스마트 조명 - 2) CSR 방식: 웹 서버 구현

❑ CSR Client-Side Rendering 방식으로 라즈베리파이 웹 서버 구현

- **CSR 방식** : 브라우저(클라이언트)가 웹 화면을 직접 만드는 방식으로, 서버는 순수한 데이터(JSON 혹은 텍스트)만 전송하는 방식
- **POST 요청** : 모든 파라미터를 요청메시지 바디에 실어 요청하는 방식

클라이언트



http://라즈베리파이 IP주소:5000

POST 요청

요청메시지 body
"color=RED&pwm_value=66"

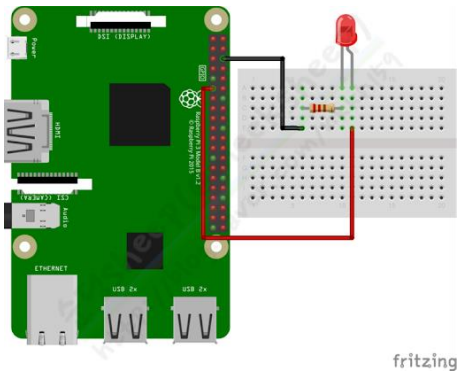
200 OK

응답메시지

"RED:66, GREEN:0, BLUE:0"

Flask 웹 서버
(app.py)

Raspberry Pi



6. 스마트 조명 - 2) CSR 방식: 웹 서버 구현

❑ CSR Client-Side Rendering 방식으로 라즈베리파이 웹 서버 구현

- 이름_6주차_RGB_app.py 에 POST 방식 처리를 위한 라우팅 추가 구현 (완성하기)

```
# CSR방식. POST 요청에 대한 처리(앱 인벤터 제어용)
@app.route('/post', methods=['POST'], strict_slashes=False)
def post_RGB():
    # 요청메시지 바디에서 key로 value 저장
    color = request.form.get('color')
    pwm_v = request.form.get('pwm_value')

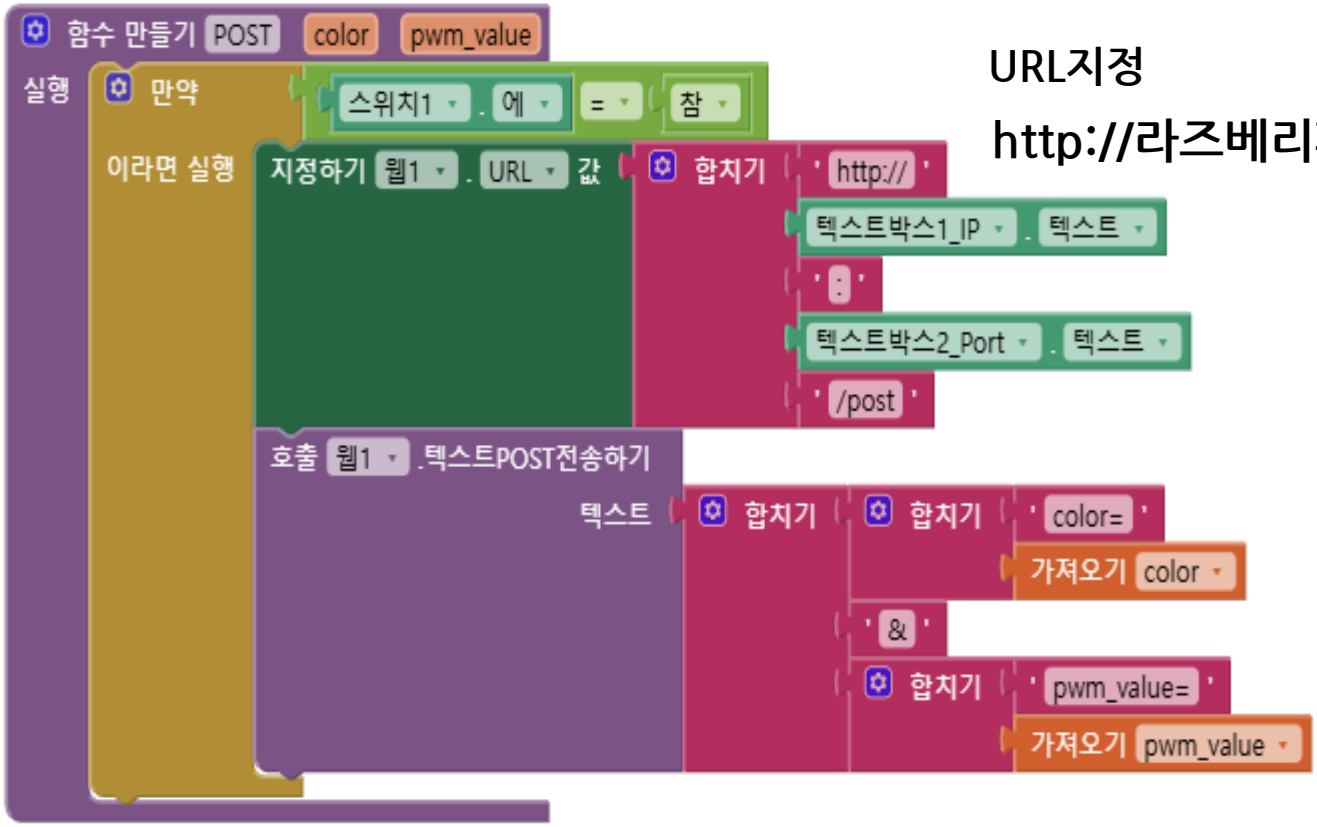
    RGB_state_dict[color] = int(pwm_v)/100
    update_RGB() # RGBLED pwm 값 업데이트

    # 앱 인벤터가 성공여부를 확인할 수 있도록 결과 문자열 반환
    return f"""RED:{RGB_state_dict['RED']},
       GREEN:{RGB_state_dict['GREEN']}
       BLUE:{RGB_state_dict['BLUE']}"""
```

6. 스마트 조명 - 2) CSR 방식: 클라이언트 앱 제작

❑ 클라이언트 앱 핵심 코드

코드동작: 섬네일의 위치가 변경되었을 때마다 [웹] POST 방식으로 파라미터 전송하기



URL지정

http://라즈베리파이IP:5000/post

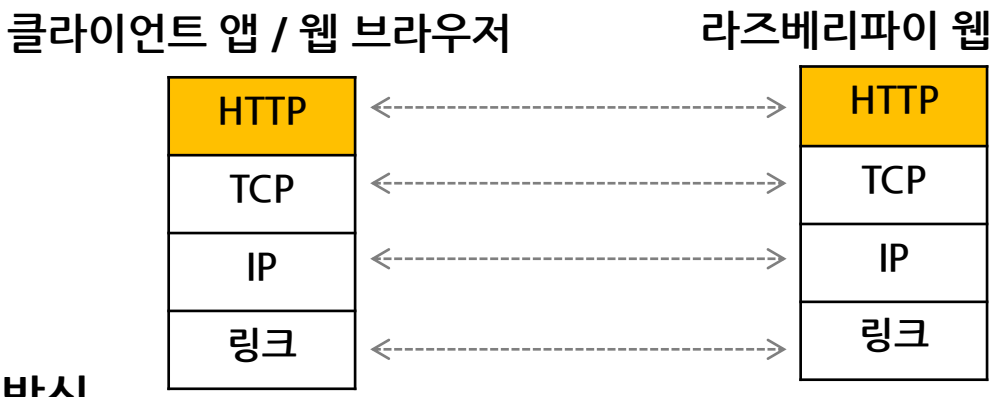
form형식의 Body 작성 후 POST 전송

바디의 form형식: “color=RED&pwm_value=90”

7. 스마트 도어락 - 동작환경

❑ 스마트 도어락 프로토콜 스택

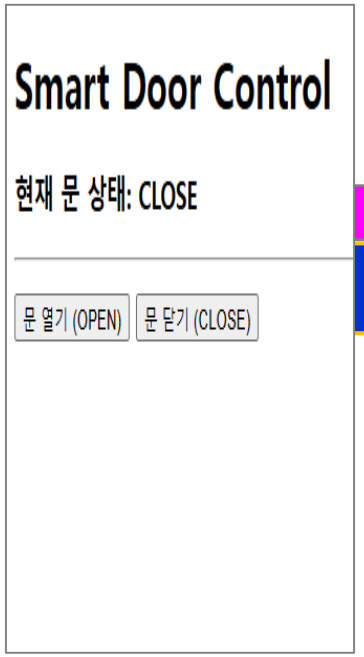
TCP/IP 프로토콜 스택

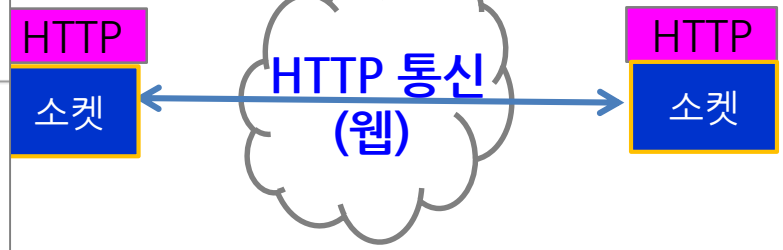


CSR방식

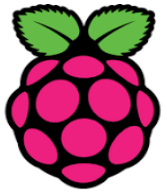


SSR방식









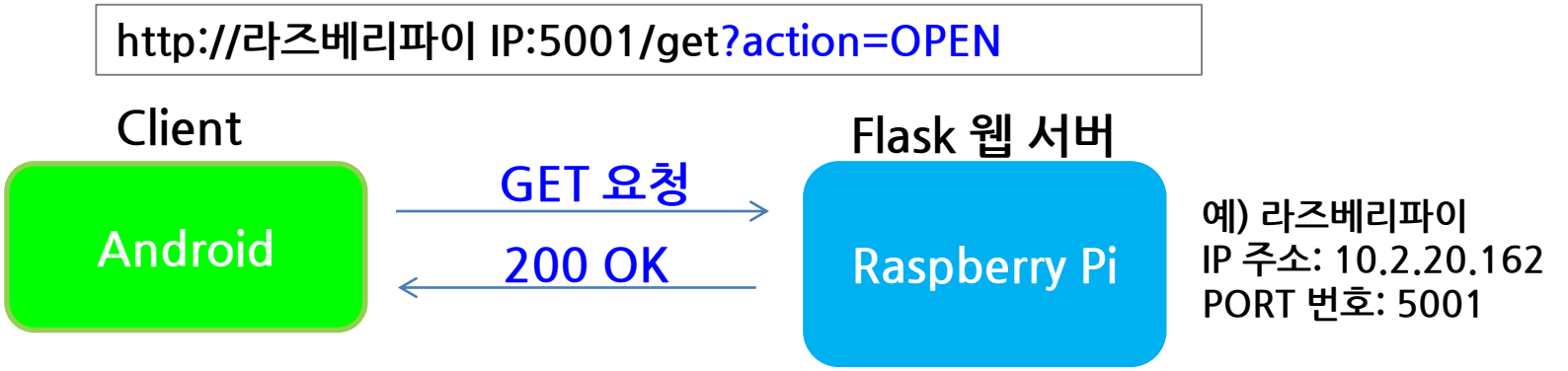
RaspberryPi

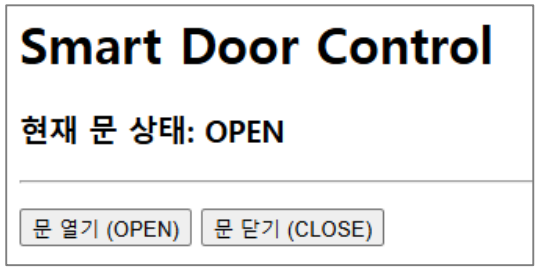
7. 스마트 도어락 - SSR 방식

❑ SSR Server-Side Rendering 방식으로 라즈베리파이 웹 서버 구현

◦ [동작 개요]

- **SSR 방식** : 서버가 데이터가 반영된 완성된 HTML 페이지를 만들어서 브라우저에 전달하는 방식
- **GET 방식 사용** : 클라이언트가 파라미터를 URL 쿼리스트링으로 보내 요청하는 방식





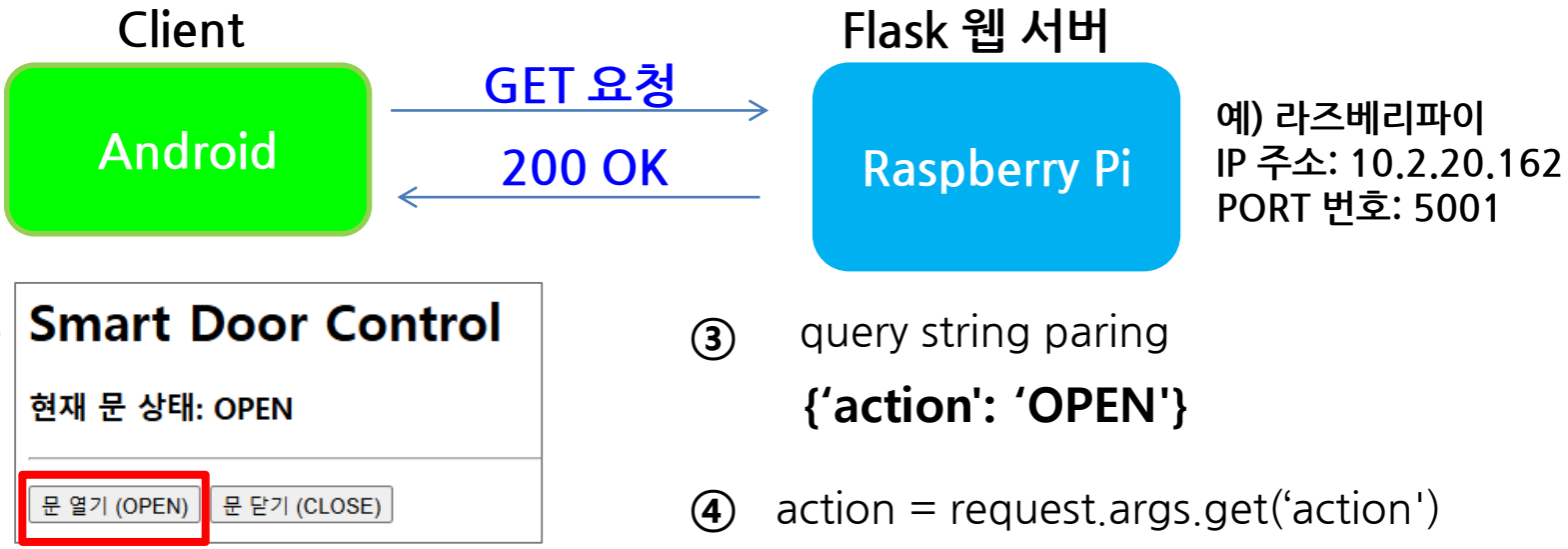
프로젝트 폴더/
├── door_app.py (Flask 서버 코드)
└── templates/
└── door_index.html (Jinja를 사용할 HTML 파일)

7. 스마트 도어락 - SSR 방식

❑ SSR Server-Side Rendering 방식으로 라즈베리파이 서버 구현 - GET 방식

◦ [동작 절차]

② http://라즈베리파이 IP:5001/get?action=OPEN



7. 스마트 도어락 - SSR 방식: 웹 서버 코드

❑ SSR Server-Side Rendering 방식으로 라즈베리파이 웹 서버 구현

◦ door_app.py

```python
# 스마트 도어락 제어 서버
from flask import Flask, request, render_template
from gpiozero import Servo
import time

# 웹 서버 객체 생성
app = Flask(__name__)

# SG90 서보모터 객체 생성 (GPIO 18번)
servo = Servo(18, min_pulse_width=0.5/1000, max_pulse_width=2.5/1000)

# door의 현재 상태를 저장할 전역변수 ('CLOSE' 또는 'OPEN')
current_status = 'CLOSE'

def control_door(action):
    global current_status
    if action == 'OPEN':
        servo.max()  # 서보모터를 최대각도(1.0)로 이동 -> 문열림
        current_status = 'OPEN'
    elif action == 'CLOSE':
        servo.min()  # 서보모터를 최소 각도(-1.0)로 이동 -> 문 닫힘
        current_status = 'CLOSE'
    # 모터가 회전할 수 있는 시간 주기
    time.sleep(0.5)
    # 이동 후 펄스신호를 끊어 떨림(jitter) 방지
    servo.detach()

# 초기 상태 설정 : 서버가 켜질때 문닫음
control_door('CLOSE')
```

```python
# 1. Flask 홈 화면 및 SSR 랜더링
@app.route('/', strict_slashes=False)
def index():
    # 현재 문 상태를 'door_status'라는 변수로 HTML로 전달
    return render_template('door_index.html', door_status=current_status)

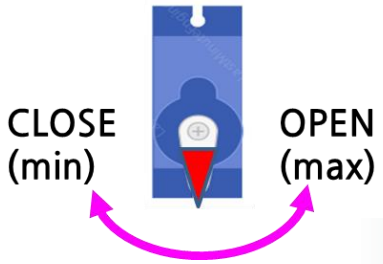
# 2. SSR방식. GET 요청 시 처리 (웹 버튼 클릭 시)
@app.route('/get', methods=['GET'], strict_slashes=False)
def get_door():
    action = request.args.get('action') # 주소창에서 ?action=값 읽기

    if action in ['OPEN', 'CLOSE']:
        control_door(action)

    return render_template('door_index.html', door_status=current_status)

if __name__ == "__main__":
    # 앞선 실습과 충돌나지 않도록 포트는 5001번으로 수정
    app.run(host='0.0.0.0', port=5001)
```

템플릿엔진으로 넘겨주는 값



7. 스마트 도어락 - SSR 방식: HTML 코드

❑ SSR Server-Side Rendering 방식으로 클라이언트로 전송할 HTML 파일 구현

◦ templates/**door_index.html**

```
<!DOCTYPE html>
<html>
<head>
    <!-- 스마트폰 화면 크기에 맞춰 웹페이지를 맞춰주는 필수 태그 -->
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>LED</title>
</head>
<body>
    <h1>Smart door Control</h1>

    <!-- 파이썬에서 넘겨받은 현재 문의 상태(문자열) 출력 -->
    <h3>현재 문 상태: {{ door_status }}</h3>
    <hr>

    <!-- GET 방식 제어(웹 브라우저 제어용) -->
    <p>
        <a href="/get?action=OPEN"><button>문 열기</button></a>
        <a href="/get?action=CLOSE"><button>문 닫기</button></a>
    </p>

</body>
</html>
```

Jinja2가 사용된 부분

7. 스마트 도어락 - CSR 방식: 라즈베리파이 웹 서버 구현

❑ CSR Client-Side Rendering 방식을 처리하도록 라즈베리파이 웹 서버 구현

◦ door_app.py에 POST 방식 처리 코드 추가 구현

```
# 3. CSR방식. POST 요청시 처리 (앱 인벤터 제어용)
@app.route('/post', methods=['POST'], strict_slashes=False)
def post_door():
    action = request.form.get('action') # POST 요청메시지 body에서 action 값 읽기
    print("POST요청", action)

    if action in ['OPEN','CLOSE']: # OPEN 혹은 CLOSE인 경우
        control_door(action)

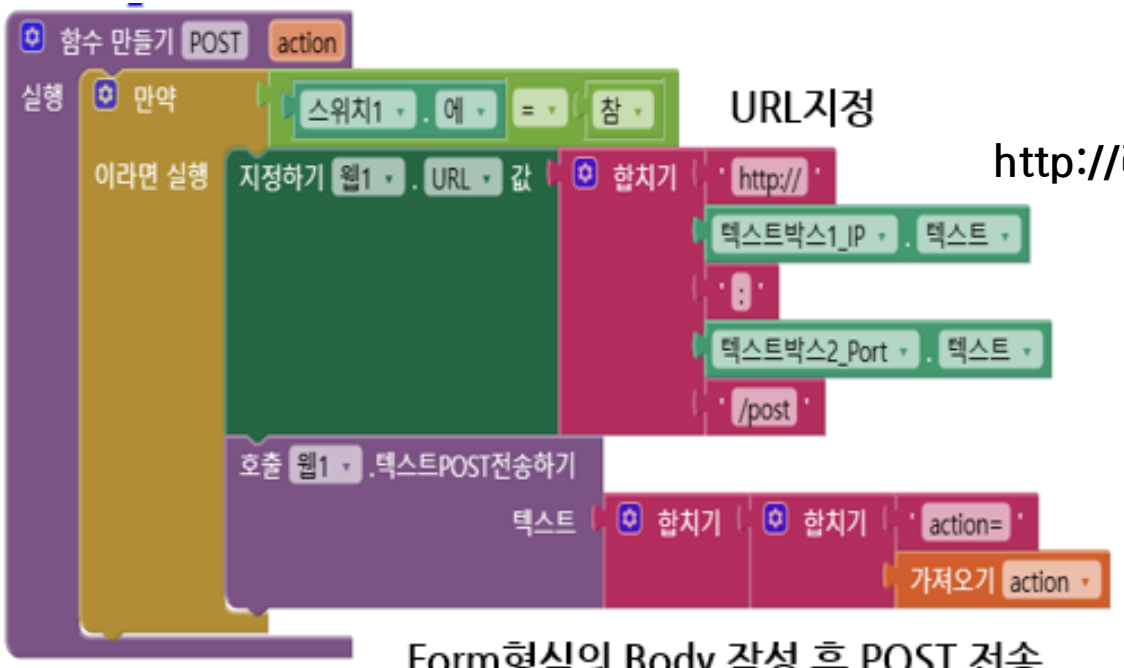
    # 앱 인벤터가 성공여부를 확인할 수 있도록 결과 문자열 반환
    return f"DOOR_STAUTS: {current_status}"

if __name__ == "__main__":
    # 앞선 실습과 충돌나지 않도록 포트는 5001번으로 수정
    app.run(host='0.0.0.0', port=5001)
```

7. 스마트 도어락 - CSR 방식: IoT 서비스 앱 제작

❑ 클라이언트 앱 핵심 코드

코드동작: Open, Close 버튼 클릭 시 POST 함수 호출



URL지정

http://라즈베리파이IP:5001/post

Form형식의 Body 작성 후 POST 전송

바디의 form형식: “action=OPEN”



수고 많았습니다!